

Universidade do Minho
Escola de Engenharia

Redes de Computadores

Trabalho Prático 1

Licenciatura em Engenharia Informática
Ano Letivo 2025/2026

Grupo 30

A106842 Gonalo Rodrigues Ribeiro
A106888 Jos  M rio Raimundo Lima
A108961 Rodrigo Novais da Silva

RC

Índice

1. Introdução	3
2. Parte 1	4
2.1. Questão 1	4
2.1.1. a)	4
2.1.2. b)	6
2.1.3. c)	7
2.2. Questão 2	8
2.2.1. a)	9
2.2.2. b)	9
2.2.3. c)	9
2.2.4. d)	9
2.2.5. e)	10
2.2.6. f)	11
2.2.7. g)	12
2.2.7.1. g.i)	12
2.2.7.2. g.ii)	12
2.3. Questão 3	13
2.3.1. a)	13
2.3.2. b)	14
2.3.3. c)	15
2.3.4. d)	16
2.3.5. e)	17
2.3.6. f)	18
2.3.7. g)	19
2.3.8. h)	19
2.3.9. i)	20
3. Parte 2	21
3.1. Questão 1	22
3.1.1. a)	22
3.1.2. b)	23
3.1.3. c)	25
3.2. Questão 2	27
3.2.1. a)	27
3.2.2. b)	28
3.2.3. c)	30
3.2.4. d)	35
3.2.4.1. i)	35
3.2.4.2. ii)	38
3.2.5. e)	40
3.2.6. f)	41
3.2.7. g)	42
3.3. Questão 3	43

4. Conclusão	47
---------------------------	-----------

1. Introdução

Este relatório descreve as atividades realizadas no âmbito do Trabalho Prático 1 da Unidade Curricular de Redes de Computadores. O objetivo central é o estudo do protocolo IPv4, abordando aspetos como a análise de cabeçalhos, o processo de fragmentação, o endereçamento e o encaminhamento em redes complexas.

Nota sobre o Endereçamento: Importa referir que, no momento da configuração das topologias, a numeração oficial dos grupos ainda não estava totalmente definida. Seguindo a orientação do docente para viabilizar o progresso do trabalho, assumiu-se o Grupo 31 (PL31) como base para todo o endereçamento IP. Assim, as redes do Core utilizam o padrão 10.31.0.xxx e os polos o padrão 192.168.31.xxx. Esta escolha garante a consistência técnica entre todos os diagramas, capturas de ecrã e tabelas de encaminhamento apresentados.

2. Parte 1

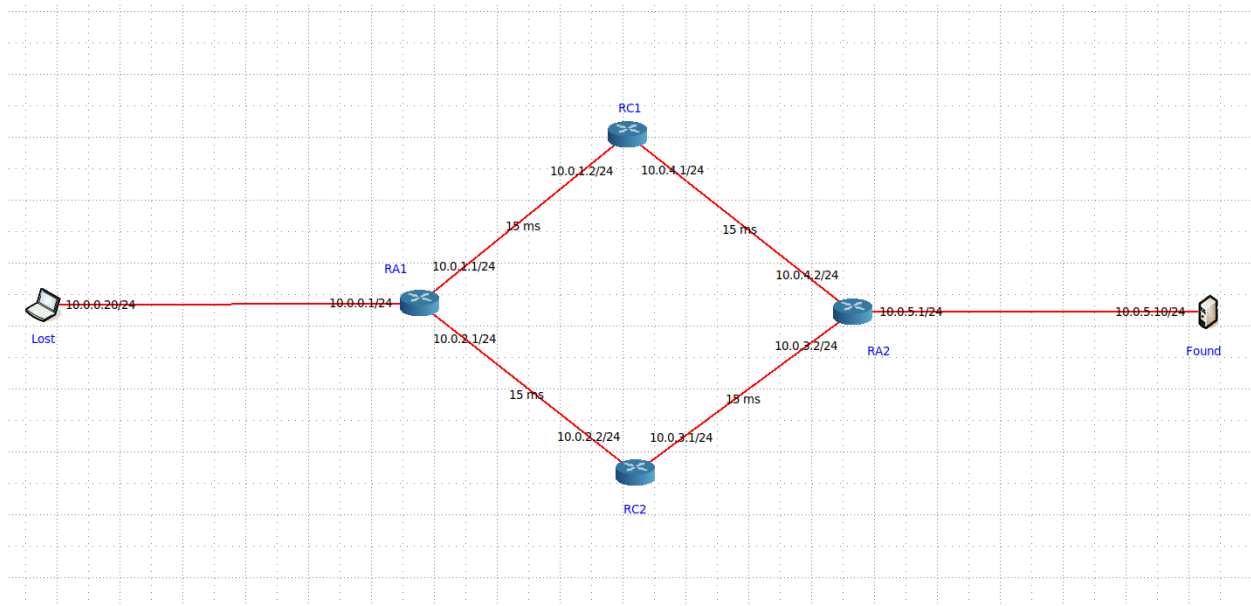


Figura 1: Topologia do Sistema proposta.

2.1. Questão 1

2.1.1. a)

Active o Wireshark no host Lost. Numa shell de Lost execute o comando `traceroute -I` para o endereço IP do Found. Registe e analise o tráfego ICMP enviado pelo sistema Lost e o tráfego ICMP recebido como resposta. Explique os resultados obtidos tendo em conta o princípio de funcionamento do traceroute.

```
vcmd
root@Lost:/tmp/pycore.36843/Lost.conf# traceroute -I 10.0.5.10
traceroute to 10.0.5.10 (10.0.5.10), 30 hops max, 60 byte packets
 1 10.0.0.1 (10.0.0.1)  0.082 ms  0.010 ms  0.008 ms
 2 10.0.1.2 (10.0.1.2)  31.055 ms  31.040 ms  31.036 ms
 3 10.0.3.2 (10.0.3.2)  62.652 ms  62.649 ms  62.645 ms
 4 10.0.5.10 (10.0.5.10)  62.641 ms  62.637 ms  62.634 ms
root@Lost:/tmp/pycore.36843/Lost.conf#
```

Figura 2: Output do comando traceroute no host Lost.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	10.0.0.1	224.0.0.5	OSPF	78	Hello Packet
2	1.802081141	fe80::3868:66ff:fec9:e932	ff02::fb	MDNS	203	Standard query 0x0000 PTR _nfs._tcp.local, "QM" question PTR _ipp._tc...
3	2.001187281	10.0.0.1	224.0.0.5	OSPF	78	Hello Packet
4	2.287575611	fe80::4c5c:23ff:fe36:47c0	ff02::fb	MDNS	203	Standard query 0x0000 PTR _nfs._tcp.local, "QM" question PTR _ipp._tc...
5	4.001986588	10.0.0.1	224.0.0.5	OSPF	78	Hello Packet
6	4.409759098	10.0.0.20	10.0.5.10	ICMP	74	Echo (ping) request id=0x0023, seq=1/256, ttl=1 (no response found!)
7	4.409812805	10.0.0.1	10.0.0.20	ICMP	102	Time-to-live exceeded (Time to live exceeded in transit)
8	4.409826813	10.0.0.20	10.0.5.10	ICMP	74	Echo (ping) request id=0x0023, seq=2/512, ttl=1 (no response found!)
9	4.409833890	10.0.0.1	10.0.0.20	ICMP	102	Time-to-live exceeded (Time to live exceeded in transit)
10	4.409838985	10.0.0.20	10.0.5.10	ICMP	74	Echo (ping) request id=0x0023, seq=3/768, ttl=1 (no response found!)
11	4.409844328	10.0.0.1	10.0.0.20	ICMP	102	Time-to-live exceeded (Time to live exceeded in transit)
12	4.409849859	10.0.0.20	10.0.5.10	ICMP	74	Echo (ping) request id=0x0023, seq=4/1024, ttl=2 (no response found!)
13	4.409869621	10.0.0.20	10.0.5.10	ICMP	74	Echo (ping) request id=0x0023, seq=5/1280, ttl=2 (no response found!)
14	4.409875944	10.0.0.20	10.0.5.10	ICMP	74	Echo (ping) request id=0x0023, seq=6/1536, ttl=2 (no response found!)
15	4.409882725	10.0.0.20	10.0.5.10	ICMP	74	Echo (ping) request id=0x0023, seq=7/1792, ttl=3 (no response found!)
16	4.409888810	10.0.0.20	10.0.5.10	ICMP	74	Echo (ping) request id=0x0023, seq=8/2048, ttl=3 (no response found!)
17	4.409894471	10.0.0.20	10.0.5.10	ICMP	74	Echo (ping) request id=0x0023, seq=9/2304, ttl=3 (no response found!)
18	4.409900616	10.0.0.20	10.0.5.10	ICMP	74	Echo (ping) request id=0x0023, seq=10/2560, ttl=4 (reply in 37)
19	4.409906217	10.0.0.20	10.0.5.10	ICMP	74	Echo (ping) request id=0x0023, seq=11/2816, ttl=4 (reply in 38)
20	4.409911723	10.0.0.20	10.0.5.10	ICMP	74	Echo (ping) request id=0x0023, seq=12/3072, ttl=4 (reply in 39)
21	4.409917986	10.0.0.20	10.0.5.10	ICMP	74	Echo (ping) request id=0x0023, seq=13/3328, ttl=5 (reply in 40)
22	4.409923438	10.0.0.20	10.0.5.10	ICMP	74	Echo (ping) request id=0x0023, seq=14/3584, ttl=5 (reply in 41)
23	4.409931906	10.0.0.20	10.0.5.10	ICMP	74	Echo (ping) request id=0x0023, seq=15/3840, ttl=5 (reply in 42)
24	4.409938956	10.0.0.20	10.0.5.10	ICMP	74	Echo (ping) request id=0x0023, seq=16/4096, ttl=6 (reply in 43)
25	4.410790508	10.0.0.20	10.0.5.10	ICMP	74	Echo (ping) request id=0x0023, seq=17/4352, ttl=6 (reply in 44)
26	4.410809055	10.0.0.20	10.0.5.10	ICMP	74	Echo (ping) request id=0x0023, seq=18/4608, ttl=6 (reply in 45)
27	4.410816448	10.0.0.20	10.0.5.10	ICMP	74	Echo (ping) request id=0x0023, seq=19/4864, ttl=7 (reply in 46)
28	4.440892895	10.0.1.2	10.0.0.20	ICMP	102	Time-to-live exceeded (Time to live exceeded in transit)
29	4.440896442	10.0.1.2	10.0.0.20	ICMP	102	Time-to-live exceeded (Time to live exceeded in transit)
Frame 18: 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface veth1.0.64, id 0						
Ethernet II, Src: 00:00:00:aa:00:00 (00:00:00:aa:00:00), Dst: 00:00:00:aa:00:01 (00:00:00:aa:00:01)						
Internet Protocol Version 4, Src: 10.0.0.20, Dst: 10.0.5.10						
0100 ... = Version: 4						
... 0101 = Header Length: 20 bytes (5)						
Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)						
Total Length: 60						
Identification: 0x4786 (18310)						
Flags: 0x0000						
Fragment offset: 0						
Time to live: 4						
Protocol: ICMP (1)						
Header checksum: 0x561e [validation disabled]						
[Header checksum status: Unverified]						
Source: 10.0.0.20						
Destination: 10.0.5.10						

Figura 3: Captura de tráfego ICMP no Wireshark.

A execução do comando `tracert -I 10.0.5.10` no host Lost permitiu descobrir a rota até ao servidor Found através do envio de pacotes **ICMP Echo Request**. Como se pode observar no output do terminal (Figura 2), foram necessários **4 saltos** para atingir o destino (**TTL = 4**).

Pela análise do Wireshark (Figura 3), confirma-se que os primeiros três pacotes enviados deram mensagens de erro **ICMP Type 11 (TTL exceeded)** enviadas pelos routers **RA1, RC1 e RA2**, respetivamente. O primeiro pacote a alcançar o destino final sem erros foi o datagrama com **TTL igual a 4**, que obteve como resposta um **ICMP Echo Reply** vindo do IP **10.0.5.10**.

2.1.2. b)

Qual deve ser o valor inicial mínimo do campo TTL para alcançar o servidor Found? Esboce um esquema com o valor do campo TTL à chegada a cada um dos routers percorridos até ao servidor Found. Verifique na prática que a sua resposta está correta.

O valor inicial mínimo do TTL é 4. Este valor é necessário porque o datagrama atravessa três routers antes de atingir o destino final. Como cada router no percurso decrementa o TTL em um, um valor inferior a 4 faria com que o pacote fosse descartado por um dos dispositivos intermédios ao atingir o valor zero.

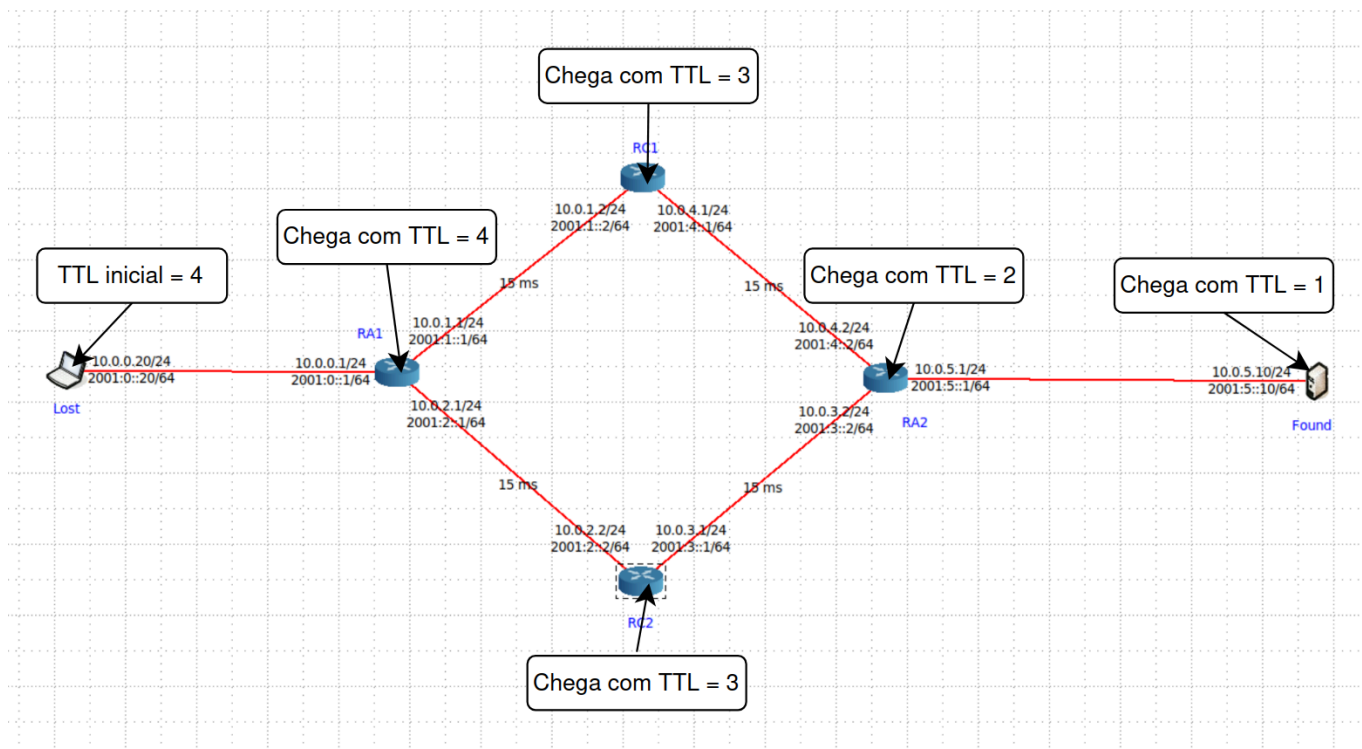
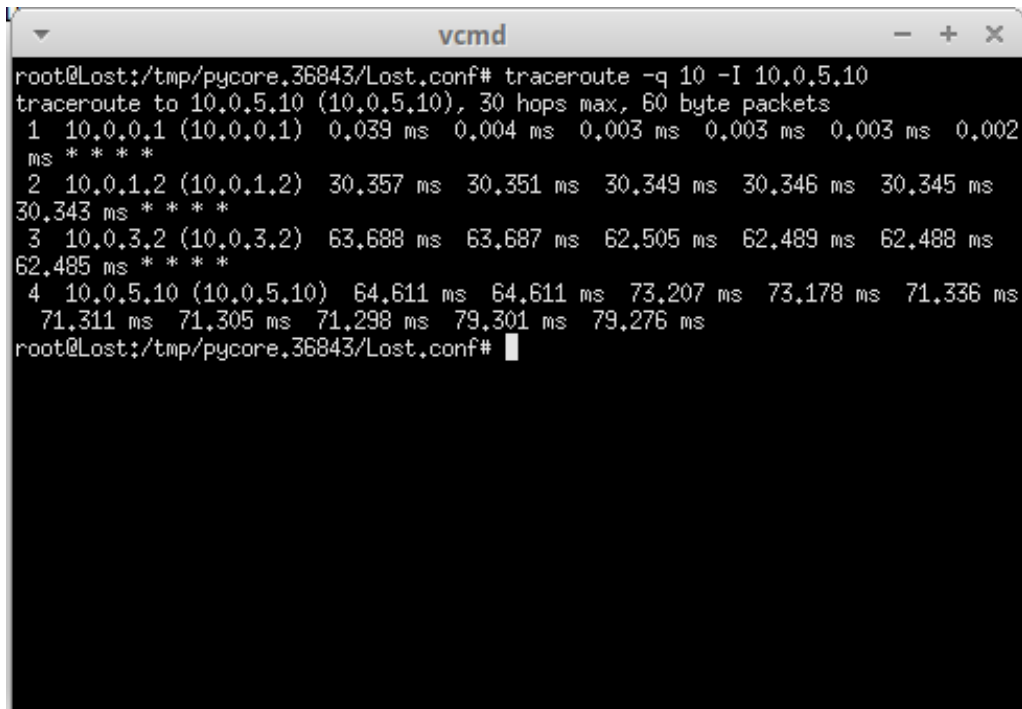


Figura 4: Esquema de variação do campo TTL à chegada de cada nó da rede.

A verificação prática deste valor é confirmada pelo output do terminal na Figura 2, onde o servidor **Found** apenas responde com sucesso no quarto salto.

2.1.3. c)

Calcule o valor médio do tempo de ida-e-volta (RTT - Round-Trip Time) obtido no acesso ao servidor. Por modo a obter uma média mais confiável, poderá alterar o número pacotes de prova com a opção -q.



```
root@Lost:/tmp/pycore.36843/Lost.conf# traceroute -q 10 -I 10.0.5.10
traceroute to 10.0.5.10 (10.0.5.10), 30 hops max, 60 byte packets
 1 10.0.0.1 (10.0.0.1)  0.039 ms  0.004 ms  0.003 ms  0.003 ms  0.003 ms  0.002
ms  * * * *
 2 10.0.1.2 (10.0.1.2)  30.357 ms  30.351 ms  30.349 ms  30.346 ms  30.345 ms
30.343 ms  * * * *
 3 10.0.3.2 (10.0.3.2)  63.688 ms  63.687 ms  62.505 ms  62.489 ms  62.488 ms
62.485 ms  * * * *
 4 10.0.5.10 (10.0.5.10)  64.611 ms  64.611 ms  73.207 ms  73.178 ms  71.336 ms
71.311 ms  71.305 ms  71.298 ms  79.301 ms  79.276 ms
root@Lost:/tmp/pycore.36843/Lost.conf#
```

Figura 5: Output do comando traceroute com envio de 10 pacotes de prova.

Observando a imagem, encontramos os seguintes pacotes com o mínimo inicial do campo TTL para alcançar o servidor **Found**.

Agora, calculamos a média:

$$RTT_{\text{médio}} = \frac{64.611 + 64.611 + 73.207 + 73.178 + 71.336 + 71.311 + 71.305 + 71.298 + 79.301 + 79.276}{10}$$

$$RTT_{\text{médio}} = 71.943 \text{ ms}$$

Portanto, o tempo médio de ida e volta (RTT médio) entre **Lost** e **Found**, baseado nos pacotes da captura, é de **71.943 ms**. Isso reflete o tempo total necessário para que os pacotes viagem do host **Lost** até o servidor **Found** e voltem.

2.2. Questão 2

```
PS C:\Users\User> tracert marco.uminho.pt

Tracing route to marco.uminho.pt [193.136.9.240]
over a maximum of 30 hops:

  1      3 ms      2 ms      1 ms     172.26.254.254
  2      3 ms      2 ms      2 ms     172.16.1.12
  3      1 ms      1 ms      2 ms     172.16.115.252
  4      6 ms      3 ms      1 ms     marco.uminho.pt [193.136.9.240]

Trace complete.
PS C:\Users\User>
```

Figura 6: Comando tracert no Windows nativo

No.	Time	Source	Destination	Protocol	Length	Info
10	3.468442000	172.26.124.87	193.136.9.240	ICMP	106	Echo (ping) request id=0x0001, seq=27/6912, ttl=1 (no response found!)
11	3.463738400	172.26.254.254	172.26.124.87	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
12	3.464999000	172.26.124.87	193.136.9.240	ICMP	106	Echo (ping) request id=0x0001, seq=28/7168, ttl=1 (no response found!)
13	3.466986100	172.26.254.254	172.26.124.87	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
14	3.468115700	172.26.124.87	193.136.9.240	ICMP	106	Echo (ping) request id=0x0001, seq=29/7424, ttl=1 (no response found!)
15	3.469764700	172.26.254.254	172.26.124.87	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
43	9.436778300	172.26.124.87	193.136.9.240	ICMP	106	Echo (ping) request id=0x0001, seq=30/7680, ttl=2 (no response found!)
44	9.448141200	172.16.1.12	172.26.124.87	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
45	9.452339400	172.26.124.87	193.136.9.240	ICMP	106	Echo (ping) request id=0x0001, seq=31/7936, ttl=2 (no response found!)
46	9.454242300	172.16.1.12	172.26.124.87	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
47	9.457828200	172.26.124.87	193.136.9.240	ICMP	106	Echo (ping) request id=0x0001, seq=32/8192, ttl=2 (no response found!)
48	9.459547600	172.16.1.12	172.26.124.87	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
169	15.286756100	172.26.124.87	193.136.9.240	ICMP	106	Echo (ping) request id=0x0001, seq=33/8448, ttl=3 (no response found!)
170	15.288274100	172.16.115.252	172.26.124.87	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
171	15.292279500	172.26.124.87	193.136.9.240	ICMP	106	Echo (ping) request id=0x0001, seq=34/8704, ttl=3 (no response found!)
172	15.293842400	172.16.115.252	172.26.124.87	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
173	15.306066700	172.26.124.87	193.136.9.240	ICMP	106	Echo (ping) request id=0x0001, seq=35/8960, ttl=3 (no response found!)

> Frame 10: Packet, 106 bytes on wire (848 bits), 106 bytes captured (848 bits) on interface \Device\NPF_{04B402CD-B100-408D-8129-A7520C6F}	0000	d4 7f 35 16 e2 9f a4 42 3b 82 d0 de 08 00 45 00	..5...B ;.....E
> Ethernet II, Src: Intel_82:d0:de (a4:42:3b:82:d0:de), Dst: Cisco_16:e2:9f (d4:7f:35:16:e2:9f)	0010	00 5c 67 f8 00 00 01 01 00 00 ac 1a 7c 57 c1 88	..G..... W..
> Internet Protocol Version 4, Src: 172.26.124.87, Dst: 193.136.9.240	0020	09 f0 08 00 f7 e3 00 01 00 1b 00 00 00 00 00	
> 0100 = Version: 4	0030	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
> 0101 = Header Length: 20 bytes (5)	0040	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
> Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)	0050	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
> Total Length: 92	0060	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
> Identification: 0x67f8 (26616)			
> 0000 = Flags: 0x0			
> ...0 0000 0000 0000 = Fragment Offset: 0			
> Time to Live: 1			
> Protocol: ICMP (1)			
> Header Checksum: 0x0000 [validation disabled]			
> [Header checksum status: Unverified]			
> Source Address: 172.26.124.87			
> Destination Address: 193.136.9.240			
> [Stream index: 3]			
> Internet Control Message Protocol			

Figura 7: Captura de tráfego ICMP no Wireshark.

2.2.1. a)

Qual é o endereço IP da interface ativa do seu computador?

O endereço IP da interface ativa do computador é **172.26.124.87**.

2.2.2. b)

Qual é o valor do campo protocol? O que permite identificar?

O valor do campo 'Protocol' é **1** (ICMP). O valor deste campo identifica qual é o protocolo que está encapsulado no campo de dados do datagrama IP. Isto indica à máquina de destino a que módulo deve ser entregue o payload para processamento, neste caso, indica que os dados devem ser passados ao módulo ICMP.

2.2.3. c)

Quanto bytes tem o cabeçalho IPv4? Quanto bytes tem o campo de dados (payload) do datagrama? Como se calcula o tamanho do payload?

O tamanho do payload de um datagrama IP calcula-se **subtraindo o tamanho do cabeçalho ao tamanho total do pacote**. Observando os detalhes do pacote no Wireshark, constata-se que o cabeçalho IPv4 tem 20 bytes. Sabendo que o tamanho total do datagrama indica 92 bytes, basta aplicar a fórmula $(92 - 20)$ para concluir que o campo de dados tem **72 bytes**.

2.2.4. d)

O datagrama IP foi fragmentado? Justifique.

O datagrama IP não foi fragmentado. Ao analisarmos o cabeçalho IPv4 no Wireshark, podemos verificar que o campo das Flags apresenta o valor **000**, estando a flag "More fragments" **desativada**, o que indica que não existem pacotes adicionais a seguir a este. Simultaneamente, o campo "Fragment Offset" também **se encontra a 0**, confirmando tratar-se de um datagrama único e inteiro que não sofreu qualquer processo de fragmentação.

2.2.5. e)

Analise a sequência de tráfego ICMP gerado a partir do endereço IP atribuído à interface da sua máquina. Para a sequência de mensagens ICMP enviadas pelo seu computador, indique que campos do cabeçalho IP variam de pacote para pacote. Justifique estas mudanças.

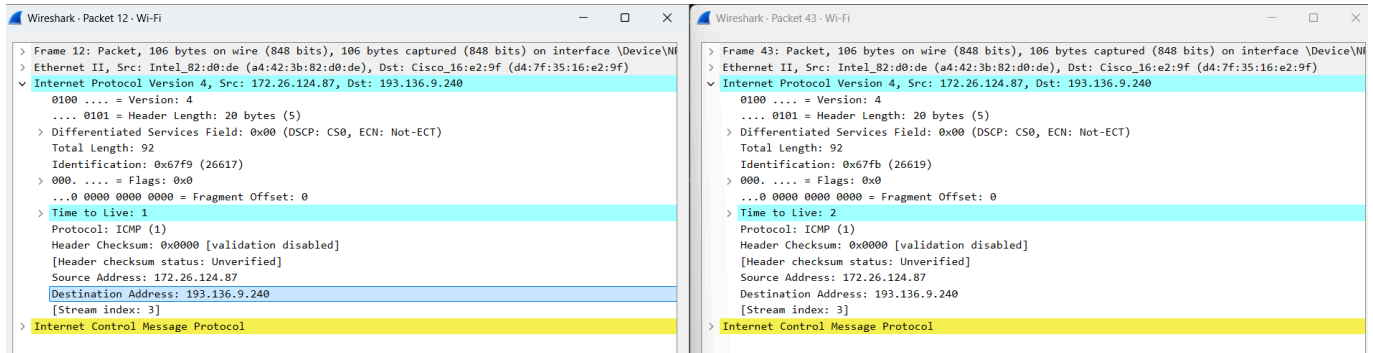


Figura 8: Dois pedidos ICMP enviados sequencialmente

Ao analisar a sequência de pedidos ICMP enviados, verificamos que os seguintes campos do cabeçalho IPv4 variam:

- **Identificação:** O valor altera-se (ex: de 26617 para 26619) porque o sistema operativo atribui um **identificador único** a cada novo pacote IP gerado, importante para a uma eventual reconstrução caso ocorra fragmentação na rede.
- **Time To Live (TTL):** O valor aumenta progressivamente. O traceroute opera enviando inicialmente datagramas com TTL a 1. Seguidamente, envia datagramas com TTL a 2, depois a 3, e assim sucessivamente, para forçar os routers no caminho a descartarem os pacotes e revelarem a sua identidade.

2.2.6. f)

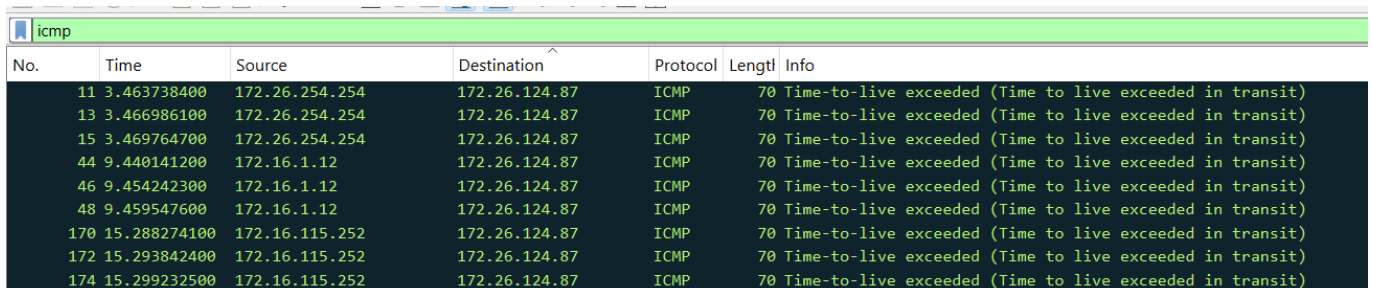
Observa algum padrão nos valores do campo de Identificação do datagrama IP e TTL?

Sim, é possível observar padrões claros em ambos os campos:

- **Padrão no TTL (Time to Live):** O valor do TTL aumenta de forma sequencial, mas em blocos. O programa tracert do Windows envia um grupo de 3 pacotes seguidos com o TTL=1. Em seguida, incrementa o valor e envia mais 3 pacotes com o TTL=2, depois mais 3 pacotes com o TTL=3, e assim sucessivamente. Este padrão serve para descobrir cada “salto” (router) da rota passo a passo.
- **Padrão na Identificação:** Os valores do campo de Identificação IP são incrementais. À medida que o sistema operativo do computador gera e envia novos datagramas, ele atribui um número de identificação único e que vai aumentando sequencialmente para cada novo pacote.

2.2.7. g)

Ordene o tráfego capturado por endereço destino e encontre a série de respostas ICMP TTL Exceeded enviadas ao seu computador.



The image shows a Wireshark packet capture window with the filter 'icmp'. The packet list shows 17 packets, all of which are ICMP 'Time-to-live exceeded' messages. The 'Destination' column is sorted, showing that the first three packets (11, 13, 15) are destined to 172.26.254.254, and the remaining 14 packets (44, 46, 48, 170, 172, 174) are destined to 172.26.124.87. All packets have a length of 70 bytes and the same information field: 'Time-to-live exceeded (Time to live exceeded in transit)'.

No.	Time	Source	Destination	Protocol	Length	Info
11	3.463738400	172.26.254.254	172.26.124.87	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
13	3.466986100	172.26.254.254	172.26.124.87	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
15	3.469764700	172.26.254.254	172.26.124.87	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
44	9.440141200	172.16.1.12	172.26.124.87	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
46	9.454242300	172.16.1.12	172.26.124.87	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
48	9.459547600	172.16.1.12	172.26.124.87	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
170	15.288274100	172.16.115.252	172.26.124.87	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
172	15.293842400	172.16.115.252	172.26.124.87	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
174	15.299232500	172.16.115.252	172.26.124.87	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)

Figura 9: Respostas ICMP TTL Exceeded

2.2.7.1. g.i)

Qual é o valor do campo TTL recebido no seu computador? Esse valor permanece constante para todas as mensagens de resposta ICMP TTL Exceeded recebidas no seu computador? Porquê?

Na primeira mensagem ICMP de erro recebida (e nas duas seguintes), o valor do campo TTL no cabeçalho IP é de 254, descendo para 253 nas seis mensagens seguintes.

Como se pode observar, este valor **não permanece constante**. Isto acontece porque, quando o TTL de um pacote de prova **atinge o valor zero**, o router **descarta o datagrama** e devolve **uma mensagem de controlo ICMP** ao host de origem, indicando que o **TTL foi excedido**. Para enviar este aviso, o router gera um datagrama IP inteiramente novo com um valor inicial de TTL predefinido e alto (neste caso, assumiu o valor de 255). Como o traceroute vai testando routers cada vez mais distantes, as respostas de erro têm de atravessar mais equipamentos no percurso de regresso. Sabendo que cada router no percurso até ao destino deve decrementar de 1 o TTL de cada datagrama recebido, o pacote de erro gerado a um salto de distância chega ao nosso computador com o TTL a 254 (255 - 1 salto), enquanto os gerados a dois saltos de distância chegam com 253, e assim sucessivamente.

2.2.7.2. g.ii)

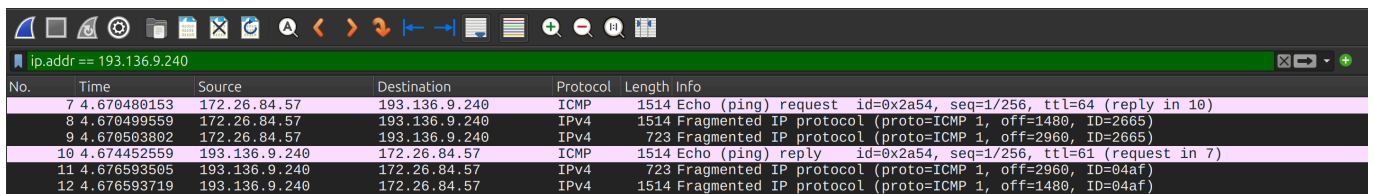
Por que razão as mensagens de resposta ICMP TTL Exceeded são sempre enviadas na origem com um valor relativamente alto?

As mensagens de erro são enviadas pelos routers com um valor inicial de TTL alto para garantir que o pacote tem “tempo de vida” suficiente para fazer **todo o caminho de regresso até ao computador de origem sem ser descartado**. Se o router gerasse o pacote de erro com um TTL muito baixo, corria-se o risco de esse próprio pacote de aviso expirar a meio do caminho de volta. Se isso acontecesse, o nosso computador nunca chegaria a ser notificado de que o pacote original se perdeu.

2.3. Questão 3

```
2: goncalo@asus-ubuntu:~  
[~]$ ping -c 1 -s 3641 marco.uminho.pt  
PING marco.uminho.pt (193.136.9.240) 3641(3669) bytes of data.  
3649 bytes from marco.uminho.pt (193.136.9.240): icmp_seq=1 ttl=61 time=6.14 ms  
  
--- marco.uminho.pt ping statistics ---  
1 packets transmitted, 1 received, 0% packet loss, time 0ms  
rtt min/avg/max/mdev = 6.139/6.139/6.139/0.000 ms
```

Figura 10: Comando ping com tamanho de dados especificado (3641 bytes).



No.	Time	Source	Destination	Protocol	Length	Info
7	4.670480153	172.26.84.57	193.136.9.240	ICMP	1514	Echo (ping) request id=0x2a54, seq=1/256, ttl=64 (reply in 10)
8	4.670499559	172.26.84.57	193.136.9.240	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=1480, ID=2665)
9	4.670503802	172.26.84.57	193.136.9.240	IPv4	723	Fragmented IP protocol (proto=ICMP 1, off=2960, ID=2665)
10	4.674452559	193.136.9.240	172.26.84.57	ICMP	1514	Echo (ping) reply id=0x2a54, seq=1/256, ttl=61 (request in 7)
11	4.676593505	193.136.9.240	172.26.84.57	IPv4	723	Fragmented IP protocol (proto=ICMP 1, off=2960, ID=04af)
12	4.676593719	193.136.9.240	172.26.84.57	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=1480, ID=04af)

Figura 11: Captura de tráfego fragmentado no Wireshark.

2.3.1. a)

Localize a primeira mensagem ICMP. Porque é que houve necessidade de fragmentar o pacote inicial?

Houve necessidade de fragmentar o pacote inicial porque o seu tamanho total **excedia o MTU da rede**, que tipicamente é de 1500 bytes para redes Ethernet.

O comando executado solicitou o envio de 3641 bytes de dados de payload ICMP. Somando a este valor os 8 bytes do cabeçalho ICMP e os 20 bytes do cabeçalho IPv4, o datagrama original teria um tamanho total de 3669 bytes. Como 3669 bytes é um valor superior ao MTU (1500 bytes), o protocolo IP na máquina de origem teve de dividir o datagrama em múltiplos fragmentos mais pequenos para que pudessem ser transmitidos pela rede.

2.3.2. b)

Imprima o primeiro fragmento do datagrama IP segmentado. Que informação no cabeçalho indica que o datagrama foi fragmentado? Que informação no cabeçalho IP indica que se trata do primeiro fragmento? Qual é o tamanho deste datagrama IP?

No.	Time	Source	Destination	Protocol	Length	Info
4	4.666134444	172.26.84.57	193.137.16.65	DNS	86	Standard query 0x8a82 AAAA marco.uminho.pt OPT
5	4.670024343	193.137.16.65	172.26.84.57	DNS	149	Standard query response 0x8a82 AAAA marco.uminho.pt SOA dns.uminho.pt OPT
6	4.670025008	193.137.16.65	172.26.84.57	DNS	102	Standard query response 0x3f3b A marco.uminho.pt A 193.136.9.240 OPT
7	4.670480153	172.26.84.57	193.136.9.240	ICMP	1514	Echo (ping) request id=0x2a54, seq=1/256, ttl=64 (reply in 10)
8	4.670499559	172.26.84.57	193.136.9.240	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=1480, ID=2665)
9	4.670503802	172.26.84.57	193.136.9.240	IPv4	723	Fragmented IP protocol (proto=ICMP 1, off=2960, ID=2665)
10	4.674452559	193.136.9.240	172.26.84.57	ICMP	1514	Echo (ping) reply id=0x2a54, seq=1/256, ttl=61 (request in 7)
11	4.676593505	193.136.9.240	172.26.84.57	IPv4	723	Fragmented IP protocol (proto=ICMP 1, off=2960, ID=04af)
12	4.676593719	193.136.9.240	172.26.84.57	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=1480, ID=04af)
13	4.677089977	172.26.84.57	193.137.16.65	DNS	97	Standard query 0x321b PTR 240.9.136.193.in-addr.arpa OPT
14	4.680354723	193.137.16.65	172.26.84.57	DNS	126	Standard query response 0x321b PTR 240.9.136.193.in-addr.arpa PTR marco.uminho.pt OPT
15	5.828184834	172.217.23.234	172.26.84.57	UDP	203	443 → 55372 Len=161
16	5.828824206	172.26.84.57	172.217.23.234	UDP	74	55372 → 443 Len=32
17	6.679350896	172.26.84.57	162.247.241.14	TLSv1.2	1313	Application Data
18	6.670750682	172.26.84.57	193.137.16.65	DNS	105	Standard query 0xd56f AAAA ham-pr-data-net-cdn.cloudflare.net OPT

Frame 7: 1514 bytes on wire (12112 bits), 1514 bytes captured (12112 bits) on interface wlo1, id 0

Ethernet II, Src: Intel_a8:d1:6c (38:7a:0e:a8:d1:6c), Dst: d4:7f:35:16:e2:9f (d4:7f:35:16:e2:9f)

Internet Protocol Version 4, Src: 172.26.84.57, Dst: 193.136.9.240

0100 = Version: 4

.... 0101 = Header Length: 20 bytes (5)

Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)

Total Length: 1500

Identification: 0x2665 (9829)

001. = Flags: 0x1, More fragments

0... = Reserved bit: Not set

.0.. = Don't fragment: Not set

..1. = More fragments: Set

...0 0000 0000 0000 = Fragment Offset: 0

Time to Live: 64

Protocol: ICMP (1)

Header Checksum: 0x62f0 [validation disabled]

[Header checksum status: Unverified]

Source Address: 172.26.84.57

Destination Address: 193.136.9.240

Internet Control Message Protocol

Figura 12: Detalhes do cabeçalho IPv4 do primeiro fragmento capturado.

Ao analisarmos o cabeçalho IPv4 do pacote selecionado (Figura 12), podemos responder às três questões:

1. **Indicação de fragmentação:** Sabemos que o pacote foi fragmentado porque no campo Flags, a flag “More fragments” está ativa.
2. **Indicação de primeiro fragmento:** O campo Fragment Offset apresenta o valor 0. Este campo indica a posição (em blocos de 8 bytes) dos dados deste fragmento em relação ao início do pacote original. Como o offset é zero e a flag “More fragments” está ativa, concluímos que este é o primeiro fragmento.
3. **Tamanho do datagrama IP:** O campo Total Length do cabeçalho IPv4 indica que o tamanho deste fragmento específico é de **1500 bytes** (precisamente o limite da MTU).

2.3.3. c)

Imprima o segundo fragmento do datagrama IP original. Que informação do cabeçalho IP indica que não se trata do 1º fragmento? Existem mais fragmentos? O que nos permite afirmar isso?

No.	Time	Source	Destination	Protocol	Length	Info
4	4.666134444	172.26.84.57	193.137.16.65	DNS	86	Standard query 0x8a82 AAAA marco.uminho.pt OPT
5	4.670024343	193.137.16.65	172.26.84.57	DNS	149	Standard query response 0x8a82 AAAA marco.uminho.pt SOA dns.uminho.pt OPT
6	4.670025008	193.137.16.65	172.26.84.57	DNS	102	Standard query response 0x3f3b A marco.uminho.pt A 193.136.9.240 OPT
7	4.670480153	172.26.84.57	193.136.9.240	ICMP	1514	Echo (ping) request id=0x2a54, seq=1/256, ttl=64 (reply in 10)
8	4.670499559	172.26.84.57	193.136.9.240	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=1480, ID=2665)
9	4.670503802	172.26.84.57	193.136.9.240	IPv4	723	Fragmented IP protocol (proto=ICMP 1, off=2960, ID=2665)
10	4.674452559	193.136.9.240	172.26.84.57	ICMP	1514	Echo (ping) reply id=0x2a54, seq=1/256, ttl=61 (request in 7)
11	4.676593505	193.136.9.240	172.26.84.57	IPv4	723	Fragmented IP protocol (proto=ICMP 1, off=2960, ID=04af)
12	4.676593719	193.136.9.240	172.26.84.57	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=1480, ID=04af)
13	4.677089977	172.26.84.57	193.137.16.65	DNS	97	Standard query 0x321b PTR 240.9.136.193.in-addr.arpa OPT
14	4.680354723	193.137.16.65	172.26.84.57	DNS	126	Standard query response 0x321b PTR 240.9.136.193.in-addr.arpa PTR marco.uminho.pt OPT
15	5.828184834	172.217.23.234	172.26.84.57	UDP	203	443 → 55372 Len=161
16	5.828824206	172.26.84.57	172.217.23.234	UDP	74	55372 → 443 Len=32
17	6.679350896	172.26.84.57	162.247.241.14	TLSv1.2	1313	Application Data
18	6.670750682	172.26.84.57	193.137.16.65	DNS	105	Standard query 0xd56f AAAA ham-pr-data-net-cdn.cloudflare.net OPT

Frame 8: 1514 bytes on wire (12112 bits), 1514 bytes captured (12112 bits) on interface wlo1, id 0
Ethernet II, Src: Intel_a8:d1:6c (38:7a:0e:a8:d1:6c), Dst: d4:7f:35:16:e2:9f (d4:7f:35:16:e2:9f)
Internet Protocol Version 4, Src: 172.26.84.57, Dst: 193.136.9.240
0100 = Version: 4
.... 0101 = Header Length: 20 bytes (5)
Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
Total Length: 1500
Identification: 0x2665 (9829)
001. = Flags: 0x1, More fragments
0... = Reserved bit: Not set
0... = Don't fragment: Not set
...1. = More fragments: Set
...0 0000 1011 1001 = Fragment Offset: 1480
Time to Live: 64
Protocol: ICMP (1)
Header Checksum: 0x6237 [validation disabled]
[Header checksum status: Unverified]
Source Address: 172.26.84.57
Destination Address: 193.136.9.240
Data (1480 bytes)

Figura 13: Detalhes do cabeçalho IPv4 do segundo fragmento capturado.

Ao analisarmos o cabeçalho IPv4 do pacote selecionado (Figura 12), podemos responder às questões:

1. **Como sabemos que não é o 1º fragmento:** O campo **Fragment Offset** apresenta agora o valor **1480** (ao contrário do valor 0 do fragmento inicial). Isto indica que os dados transportados por este pacote começam na posição correspondente ao byte 1480 do payload do datagrama original. Como o offset é diferente de zero, é impossível ser o primeiro fragmento.
2. **Existem mais fragmentos:** Sim, existem mais fragmentos. Podemos afirmar isto com toda a certeza porque a flag **More fragments** continua com o valor **1 (Set)**.

2.3.4. d)

Quantos fragmentos foram criados a partir do datagrama original? Como se detecta o último fragmento correspondente ao datagrama original? Estabeleça um filtro no Wireshark que permita listar apenas o último fragmento do primeiro datagrama IP segmentado.

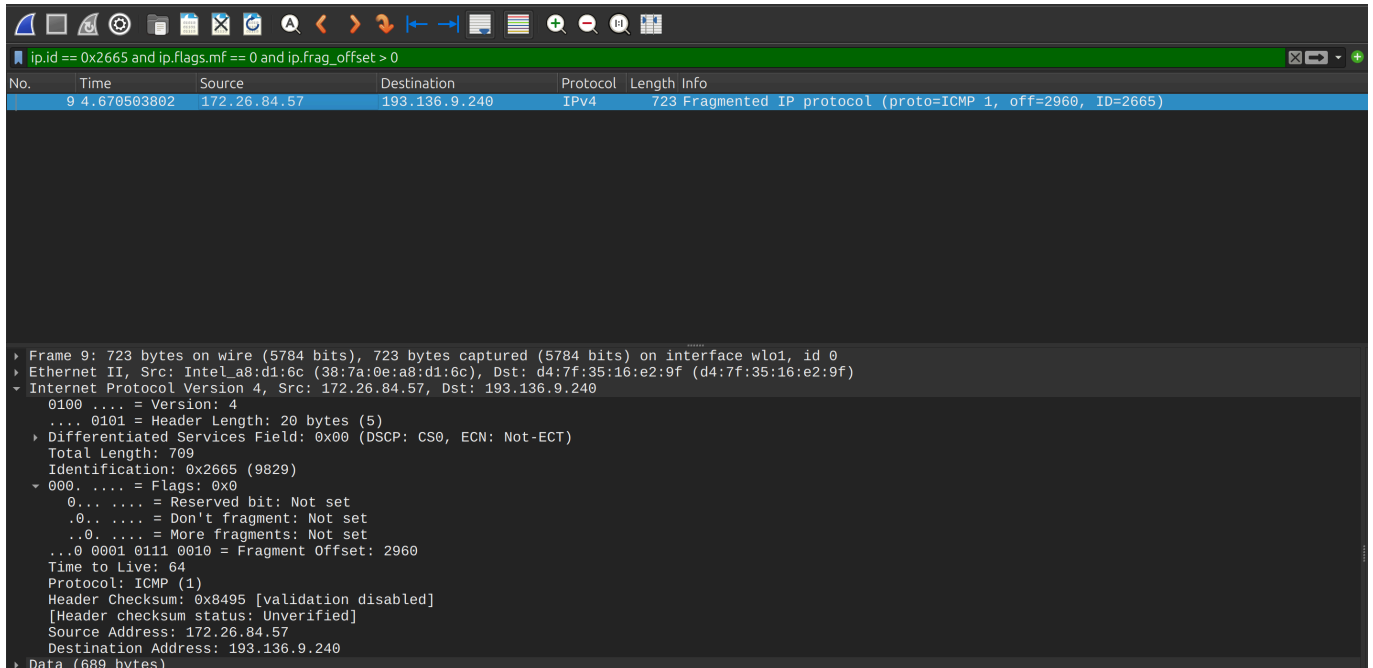


Figura 14: Filtro aplicado para capturar o último fragmento do datagrama.

Foram criados **3 fragmentos** a partir do datagrama original. O último fragmento detecta-se através da análise combinada de dois campos no cabeçalho IPv4: a flag **More fragments** tem de estar a **0 (Not set)**, indicando que não há mais pacotes a seguir a este, e o campo **Fragment Offset** tem de ser **maior que 0** (neste caso, é 2960), provando que se trata de uma continuação e não de um pacote novo não-fragmentado.

O filtro estabelecido no Wireshark (ver Figura 14) foi:

```
ip.id == 0x2665 and ip.flags.mf == 0 and ip.frag_offset > 0
```

2.3.5. e)

Indique, resumindo, os campos que mudam no cabeçalho IP entre os diferentes fragmentos, e explique a forma como essa informação permite reconstruir o datagrama original.

Entre os diferentes fragmentos, os seguintes campos do cabeçalho IPv4 sofrem alterações:

- **Total Length:** Varia consoante o tamanho do fragmento (os dois primeiros têm 1500 bytes, o último tem 709 bytes).
- **Flags (More fragments):** Está ativa (1) em todos os fragmentos exceto no último (0).
- **Fragment Offset:** Aumenta progressivamente (0, 1480 e 2960), indicando a posição relativa dos dados de cada fragmento.
- **Header Checksum:** Altera-se em cada pacote, visto que os campos anteriores também mudam, obrigando a recalcular a soma de verificação.

Reconstrução do datagrama: A máquina de destino utiliza o campo **Identification** (que é idêntico em todos os fragmentos, neste caso 0x2665) para saber que todos aqueles pedaços pertencem à mesma mensagem original. De seguida, utiliza o **Fragment Offset** para ordenar os dados transportados na sequência correta. O recetor sabe que a mensagem está completa quando recebe o fragmento com a flag **More fragments a 0** e verifica que não faltam blocos de dados entre o offset 0 e o fim do último fragmento.

2.3.6. f)

Estime teoricamente o número de fragmentos gerados e o número de bytes transportados em cada um dos fragmentos. Apresente todos os cálculos efetuados, incluindo os campos do cabeçalho IP relevantes para cada um dos fragmentos.

Dados Iniciais (Exercício de Fragmentação): Para enviar um ping com 3641 bytes de dados, temos de considerar os cabeçalhos adicionados pelas camadas inferiores.

- Dados solicitados (ping -s 3641): 3641 bytes
- Cabeçalho ICMP: 8 bytes
- Payload Total do IP: $3641 + 8 = 3649$ bytes
- Tamanho do Cabeçalho IPv4 (HL): 20 bytes
- Tamanho Total do Datagrama IP Original (L): $3649 + 20 = 3669$ bytes
- Maximum Transmission Unit (MTU): 1500 bytes
- Payload máximo por fragmento ($MTU - HL$): $1500 - 20 = 1480$ bytes

Cálculos por Fragmento:

Fragmento 1:

- $HL = 20$ bytes
- Payload = 1480 bytes
- $L = 1480 + 20 = 1500$ bytes
- Flags = **1** (More Fragments = Set)
- Fragment Offset = **0**
- *Restante a enviar:* $3649 - 1480 = 2169$ bytes

Fragmento 2:

- $HL = 20$ bytes
- Payload = 1480 bytes
- $L = 1480 + 20 = 1500$ bytes
- Flags = **1** (More Fragments = Set)
- Fragment Offset = $\frac{1480}{8} = \mathbf{185}$ (representa a posição aos 1480 bytes)
- *Restante a enviar:* $2169 - 1480 = 689$ bytes

Fragmento 3 (Último):

- $HL = 20$ bytes
- Payload = 689 bytes
- $L = 689 + 20 = 709$ bytes
- Flags = **0** (More Fragments = Not Set)
- Fragment Offset = $\frac{2960}{8} = \mathbf{370}$ (representa a posição aos 2960 bytes)
- *Restante a enviar:* $689 - 689 = 0$ bytes (Fim da transmissão)

Esquema Lógico da Distribuição (Payload IP de 3649 bytes):

F1 (1480 bytes)	F2 (1480 bytes)	F3 (689 bytes)
Offset: 0	Offset: 1480	Offset: 2960

Como se pode constatar, os valores calculados teoricamente batem exatamente certo com as capturas registadas no Wireshark (pacotes com Length de 1500, 1500 e 709 bytes, respetivamente).

2.3.7. g)

Por que razão apenas o primeiro fragmento de cada pacote é identificado pelo Wireshark como sendo um pacote ICMP? Justifique a sua resposta com base no conceito de Fragmentação apresentado nas aulas teóricas.

Uma vez que o **protocolo ICMP** é encapsulado no **datagrama IP**, a identificação da mensagem ocorre exclusivamente no **primeiro fragmento**, sendo este o único a transportar o **cabeçalho ICMP completo**. Os fragmentos subsequentes contêm apenas o **payload**. Consequentemente, o analisador de tráfego **Wireshark** não dispõe dos dados necessários para classificar isoladamente esses fragmentos adicionais como mensagens ICMP, identificando-os apenas como **tráfego IPv4 fragmentado**.

2.3.8. h)

Com que valor é o tamanho do datagrama comparado a fim de se determinar se este deve ser fragmentado? Quais seriam os efeitos na rede ao aumentar/diminuir este valor?

O tamanho total do datagrama IP é comparado com o valor da **MTU** da interface de rede de saída.

- **Aumentar a MTU:** Menos fragmentação, melhor desempenho.
- **Diminuir a MTU:** Mais fragmentação, aumento de overhead e latência.

2.3.9. i)

Sabendo que no comando ping a opção “-M do” (Linux) ativa a flag “Don’t Fragment” (DF) no cabeçalho do IPv4... determine o valor máximo de SIZE sem que ocorra fragmentação do pacote? Justifique o valor obtido.

```
1: goncalo@asus-ubuntu:~  
[~]$ ping -c 1 -M do -s 1472 marco.uminho.pt  
PING marco.uminho.pt (193.136.9.240) 1472(1500) bytes of data.  
1480 bytes from marco.uminho.pt (193.136.9.240): icmp_seq=1 ttl=61 time=5.76 ms  
  
--- marco.uminho.pt ping statistics ---  
1 packets transmitted, 1 received, 0% packet loss, time 0ms  
rtt min/avg/max/mdev = 5.763/5.763/5.763/0.000 ms  
[~]$  
  
2: goncalo@asus-ubuntu:~  
[~]$ ping -c 1 -M do -s 1473 marco.uminho.pt  
PING marco.uminho.pt (193.136.9.240) 1473(1501) bytes of data.  
ping: local error: message too long, mtu=1500  
  
--- marco.uminho.pt ping statistics ---  
1 packets transmitted, 0 received, +1 errors, 100% packet loss, time 0ms  
[~]$
```

Figura 15: Teste do tamanho máximo de payload com a flag “Don’t Fragment” ativa.

O valor máximo do parâmetro **SIZE** (os dados de payload do ping) sem que ocorra fragmentação é de **1472 bytes**.

Justificação: Sabemos que o limite máximo de transmissão na interface (MTU) é de 1500 bytes. Para calcularmos quanto espaço sobra para os dados do utilizador, temos de subtrair o tamanho dos cabeçalhos que são adicionados obrigatoriamente:

- MTU da interface: 1500 bytes
- Cabeçalho IPv4: - 20 bytes
- Cabeçalho ICMP: - 8 bytes
- **Espaço restante para SIZE:** $1500 - 20 - 8 = 1472$ bytes

Como demonstrado na Figura 15, ao executarmos o comando com o tamanho exato de 1472 bytes, o pacote é enviado e recebido com sucesso. No entanto, ao tentarmos enviar 1473 bytes com a flag **DF** ativa, o sistema operativo descarta imediatamente o pacote antes mesmo de o enviar para a rede, devolvendo o erro `message too long, mtu=1500`. Isto confirma na prática que, ao ultrapassar os 1472 bytes de dados, a fragmentação torna-se estritamente necessária.

3. Parte 2

Nota introdutória: Como o número do grupo ainda não havia sido definido pelo professor no momento da realização da componente prática deste trabalho, assumimos o grupo 31 para efeitos de cálculo de endereçamento e configuração das topologias. Para garantir a total consistência entre o novo esquema de endereçamento – transição de 192.168.0.x para 192.168.31.x nos polos e de 10.0.0.x para 10.31.0.x no Core – e assegurar a integridade das rotas estáticas pré-existentes, optou-se pela edição direta do ficheiro de configuração da topologia (.imn). Através de um editor de texto e com o auxílio da função ‘Replace All’, foi possível atualizar globalmente todos os saltos e interfaces, permitindo o avanço imediato para a análise das falhas de encaminhamento propostas no enunciado.

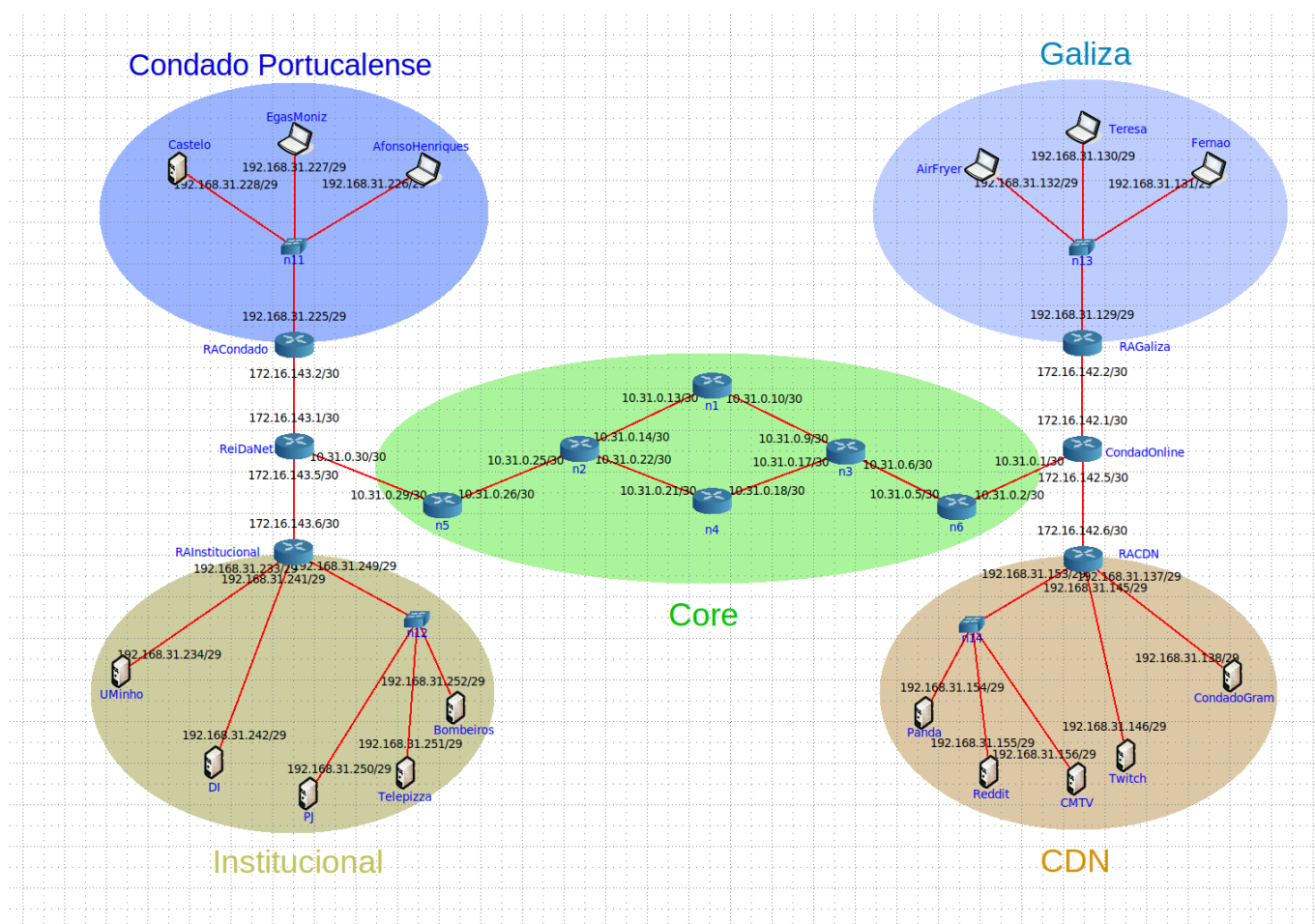


Figura 16: Topologia Global da Rede (Parte 2) atualizada com o identificador do Grupo 31.

3.1. Questão 1

3.1.1. a)

De modo a garantir uma posição estrategicamente mais vantajosa e ter casa de férias para relaxar entre batalhas, ordena a construção de um segundo Castelo, em Braga. Não tendo qualquer queixa do serviço prestado, recorre aos serviços do ISP ReiDaNet, que já utiliza no condado, para ter acesso à rede no segundo Castelo. O ISP atribuiu-lhe o endereço de rede IP 172.26.XX.128/25 em que XX corresponde ao seu número de grupo (PLXX). Defina um esquema de endereçamento que permita o estabelecimento de pelo menos 6 redes e que garanta que cada uma destas possa ter, pelo menos, 5 hosts. Assuma que todos os endereços de sub-redes são utilizáveis. Assim que concluído, chame o docente e explique o processo, justificando as suas opções

Usando como número do grupo o **31**, a rede atribuída pelo ISP é a **172.26.31.128/25**. Esta rede original possui 128 endereços IP (2^{32-25}).

Como o objetivo é criar, pelo menos, 6 sub-redes e garantir que cada uma suporta um mínimo de 5 hosts, efetuou-se o seguinte raciocínio:

1. Para acomodar 5 hosts, precisamos de, pelo menos, 3 bits dedicados aos hosts ($2^3 - 2 = 6$ endereços utilizáveis).
2. Tendo 3 bits para os hosts, a porção de rede passará a ocupar 29 bits ($32 - 3 = 29$). A nova máscara a utilizar será, portanto, **/29** (ou seja, 255.255.255.248).
3. Ao passarmos de uma máscara /25 para /29, “pedimos emprestados” 4 bits para a criação de sub-redes, o que permite gerar um total de $2^4 = 16$ sub-redes distintas. Este valor satisfaz o requisito mínimo de 6 sub-redes exigido no enunciado.

Esquema de Endereçamento Resultante: O espaço de endereçamento /25 foi segmentado em blocos de 8 IPs (máscara /29), resultando no seguinte esquema (apresentam-se as primeiras 8 das 16 sub-redes geradas):

Sub-rede	Endereço de Rede	Intervalo Útil
1	172.26.31.128/29	172.26.31.129 - 172.26.31.134
2	172.26.31.136/29	172.26.31.137 - 172.26.31.142
3	172.26.31.144/29	172.26.31.145 - 172.26.31.150
4	172.26.31.152/29	172.26.31.153 - 172.26.31.158
5	172.26.31.160/29	172.26.31.161 - 172.26.31.166
6	172.26.31.168/29	172.26.31.169 - 172.26.31.174
7	172.26.31.176/29	172.26.31.177 - 172.26.31.182
8	172.26.31.184/29	172.26.31.185 - 172.26.31.190

3.1.2. b)

Ligue um novo host Castelo2 diretamente ao router ReiDaNet. Associe-lhe um endereço, à sua escolha, pertencente a uma sub-rede disponível das criadas na alínea anterior (garanta que a interface do router ReiDaNet utiliza o primeiro endereço válido da sub-rede escolhida). Verifique que tem conectividade com os dispositivos do Condado Portucalense.

Para o Router (ReiDaNet):

- IP: 172.26.31.129/29 (Primeiro endereço válido).

Para o novo PC (Castelo2):

- IP: 172.26.31.130/29 (Segundo endereço válido).
- Gateway (Porta de saída): 172.26.31.129 (IP da interface do router).

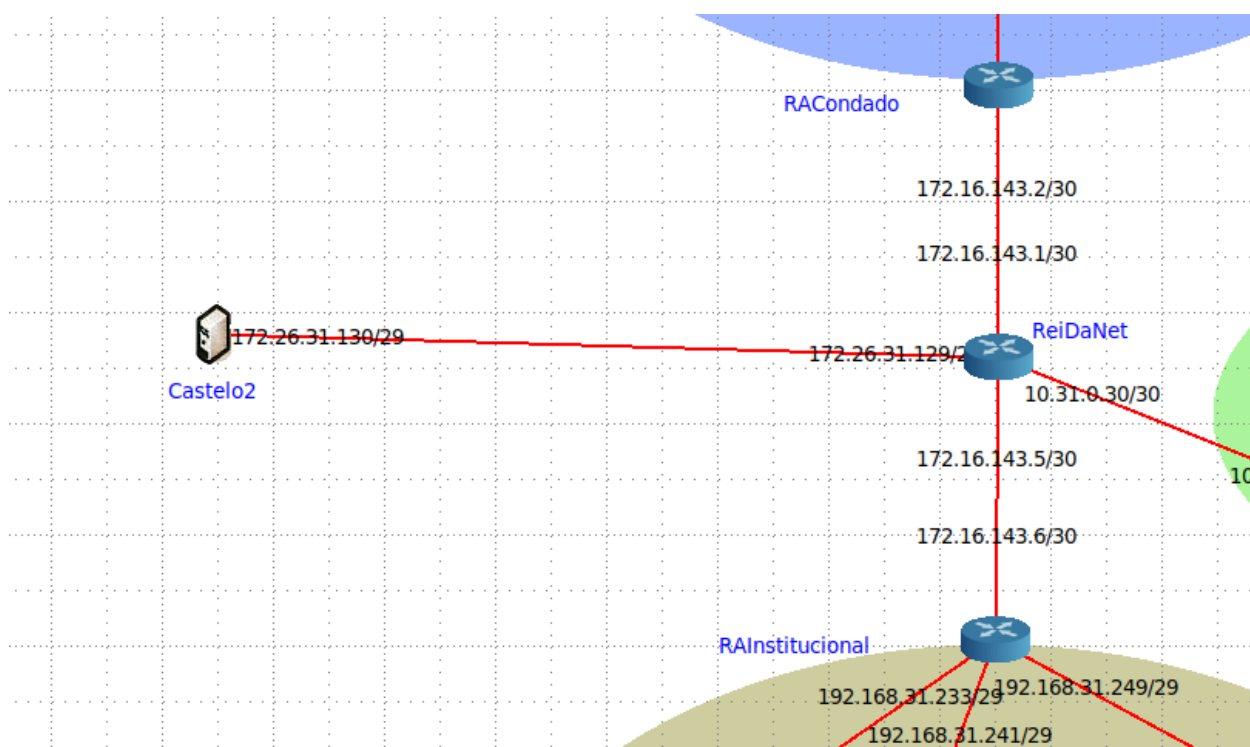
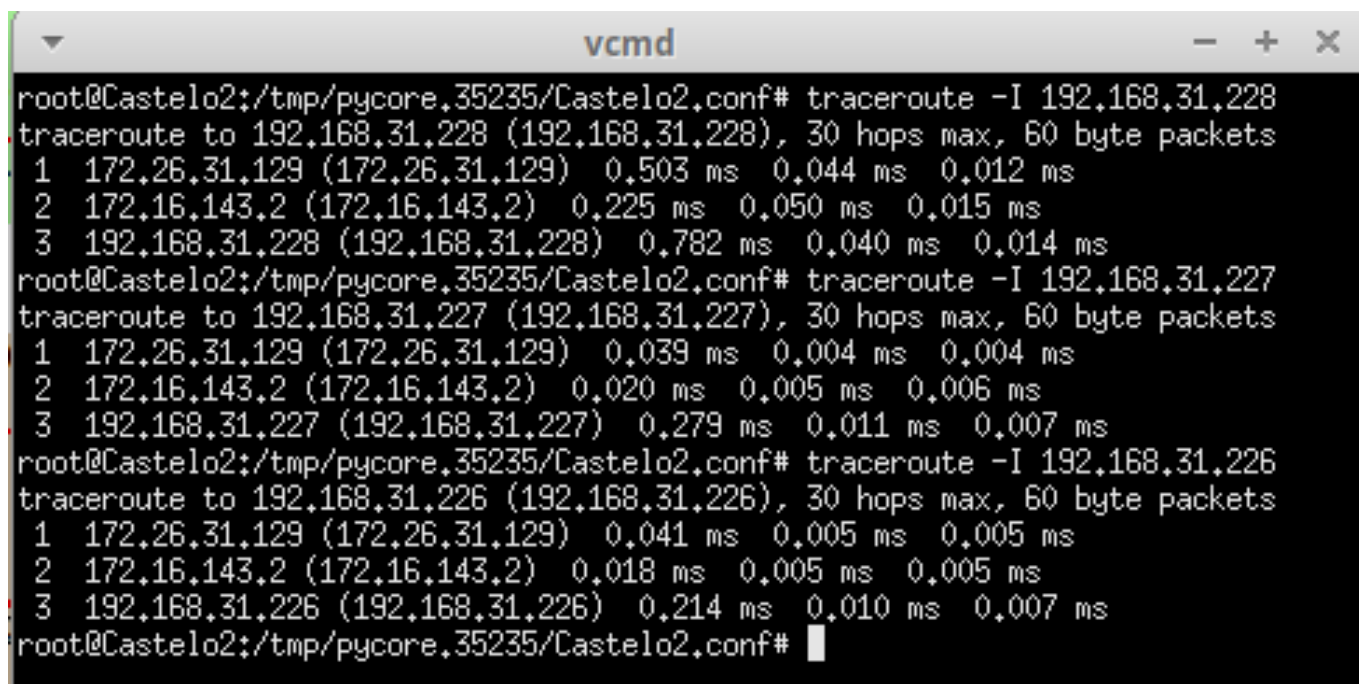


Figura 17: Ligação física e configuração de IPs do Castelo2.



```
vcmd
root@Castelo2:/tmp/pycore.35235/Castelo2.conf# traceroute -I 192.168.31.228
traceroute to 192.168.31.228 (192.168.31.228), 30 hops max, 60 byte packets
 1 172.26.31.129 (172.26.31.129) 0.503 ms 0.044 ms 0.012 ms
 2 172.16.143.2 (172.16.143.2) 0.225 ms 0.050 ms 0.015 ms
 3 192.168.31.228 (192.168.31.228) 0.782 ms 0.040 ms 0.014 ms
root@Castelo2:/tmp/pycore.35235/Castelo2.conf# traceroute -I 192.168.31.227
traceroute to 192.168.31.227 (192.168.31.227), 30 hops max, 60 byte packets
 1 172.26.31.129 (172.26.31.129) 0.039 ms 0.004 ms 0.004 ms
 2 172.16.143.2 (172.16.143.2) 0.020 ms 0.005 ms 0.006 ms
 3 192.168.31.227 (192.168.31.227) 0.279 ms 0.011 ms 0.007 ms
root@Castelo2:/tmp/pycore.35235/Castelo2.conf# traceroute -I 192.168.31.226
traceroute to 192.168.31.226 (192.168.31.226), 30 hops max, 60 byte packets
 1 172.26.31.129 (172.26.31.129) 0.041 ms 0.005 ms 0.005 ms
 2 172.16.143.2 (172.16.143.2) 0.018 ms 0.005 ms 0.005 ms
 3 192.168.31.226 (192.168.31.226) 0.214 ms 0.010 ms 0.007 ms
root@Castelo2:/tmp/pycore.35235/Castelo2.conf#
```

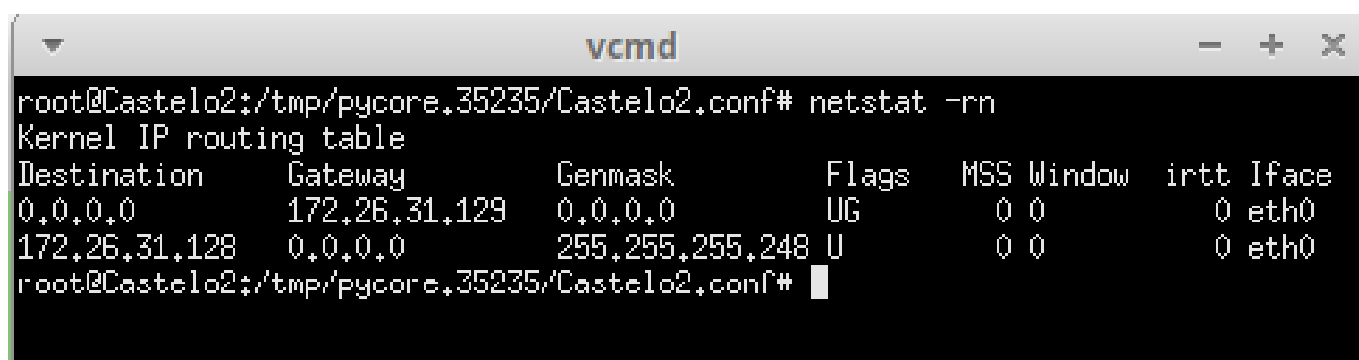
Figura 18: Testes de conectividade para os três hosts do Condado Portucalense.

Para testar a conectividade com o Condado Portucalense, executaram-se comandos `traceroute -I ...` a partir do Castelo2 com destino aos três hosts da rede original: Castelo (**192.168.31.228**), EgasMoniz (**192.168.31.227**) e AfonsoHenriques (**192.168.31.226**). Como é possível observar na Figura 18, a comunicação foi estabelecida com sucesso para todos os destinos, saltando pelo router ReiDaNet e pelo RACondado até chegar aos respectivos PCs. Isto comprova que o roteamento entre a nova sub-rede e todo o polo do Condado Portucalense se encontra perfeitamente funcional.

3.1.3. c)

Não estando satisfeito com a decoração deste novo Castelo, opta por eliminar a sua rota default. Adicione as rotas necessárias para que o Castelo2 continue a ter acesso ao Condado Portucalense e à rede Institucional. Mostre que a conectividade é restabelecida, assim como a tabela de encaminhamento resultante. Explícite ainda a utilidade de uma rota default.

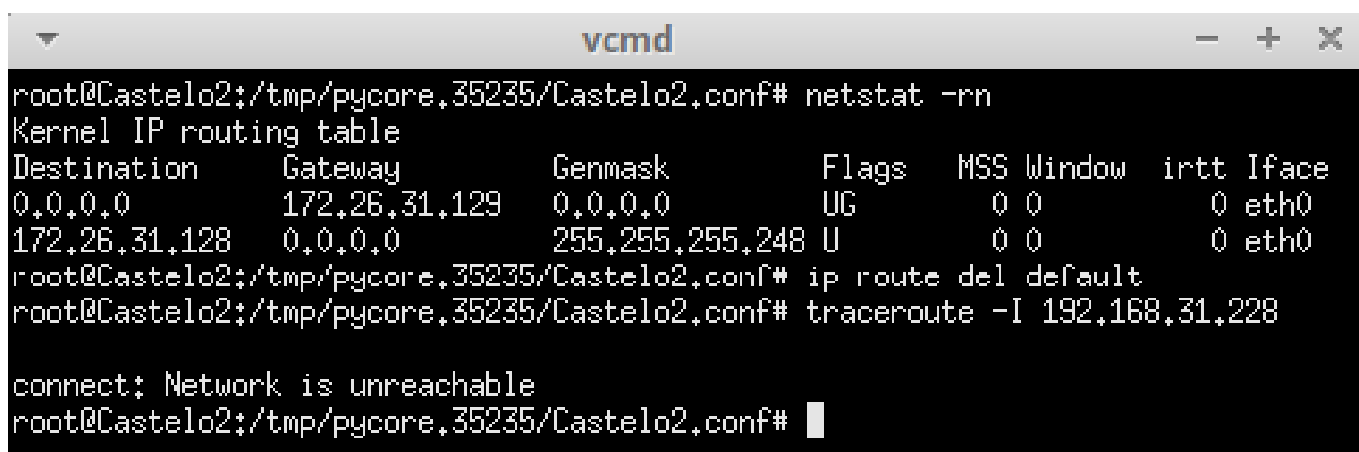
Para dar resposta a esta alínea, começou-se por verificar a tabela de encaminhamento inicial do **Castelo2** através do comando `netstat -rn`, confirmando-se a existência da rota default.



```
vcmd
root@Castelo2:/tmp/pycore.35235/Castelo2.conf# netstat -rn
Kernel IP routing table
Destination      Gateway          Genmask          Flags      MSS Window  irtt Iface
0.0.0.0          172.26.31.129   0.0.0.0          UG          0 0        0 eth0
172.26.31.128    0.0.0.0         255.255.255.248 U           0 0        0 eth0
root@Castelo2:/tmp/pycore.35235/Castelo2.conf#
```

Figura 19: Verificação da tabela de encaminhamento inicial.

De seguida, eliminou-se a rota por defeito utilizando o comando `ip route del default` e testou-se a conectividade. Como se pode constatar na figura abaixo, o sistema operativo devolveu o erro `connect: Network is unreachable`, provando que o computador perdeu a capacidade de enviar pacotes para fora da sua sub-rede local.



```
vcmd
root@Castelo2:/tmp/pycore.35235/Castelo2.conf# netstat -rn
Kernel IP routing table
Destination      Gateway          Genmask          Flags      MSS Window  irtt Iface
0.0.0.0          172.26.31.129   0.0.0.0          UG          0 0        0 eth0
172.26.31.128    0.0.0.0         255.255.255.248 U           0 0        0 eth0
root@Castelo2:/tmp/pycore.35235/Castelo2.conf# ip route del default
root@Castelo2:/tmp/pycore.35235/Castelo2.conf# traceroute -I 192.168.31.228

connect: Network is unreachable
root@Castelo2:/tmp/pycore.35235/Castelo2.conf#
```

Figura 20: Eliminação da rota default e perda de conectividade.

Para restabelecer a comunicação com o **Condado Portucalense** e a rede **Institucional**, inseriram-se manualmente as quatro rotas estáticas necessárias, definindo como *gateway* a interface do router ReiDaNet (**172.26.31.129**).

```

vcmd
root@Castelo2:/tmp/pycore.35235/Castelo2.conf# ip route add 192.168.31.224/29 via 172.26.31.129
root@Castelo2:/tmp/pycore.35235/Castelo2.conf# ip route add 192.168.31.232/29 via 172.26.31.129
root@Castelo2:/tmp/pycore.35235/Castelo2.conf# ip route add 192.168.31.240/29 via 172.26.31.129
root@Castelo2:/tmp/pycore.35235/Castelo2.conf# ip route add 192.168.31.248/29 via 172.26.31.129
root@Castelo2:/tmp/pycore.35235/Castelo2.conf# 

```

Figura 21: Adição das rotas estáticas para o Condado e Institucional.

Por fim, confirmou-se a aplicação das novas rotas na tabela de encaminhamento e efetuaram-se testes de conectividade (traceroute) para um host do Condado Portucalense (192.168.31.226) e um host da rede Institucional (192.168.31.234). Ambos os testes foram concluídos com sucesso, demonstrando o restabelecimento da ligação.

```

vcmd
root@Castelo2:/tmp/pycore.35235/Castelo2.conf# netstat -rn
Kernel IP routing table
Destination      Gateway          Genmask         Flags   MSS Window  irtt Iface
172.26.31.128    0.0.0.0          255.255.255.248 U        0 0        0 eth0
192.168.31.224   172.26.31.129    255.255.255.248 UG        0 0        0 eth0
192.168.31.232   172.26.31.129    255.255.255.248 UG        0 0        0 eth0
192.168.31.240   172.26.31.129    255.255.255.248 UG        0 0        0 eth0
192.168.31.248   172.26.31.129    255.255.255.248 UG        0 0        0 eth0
root@Castelo2:/tmp/pycore.35235/Castelo2.conf# traceroute -I 192.168.31.226
traceroute to 192.168.31.226 (192.168.31.226), 30 hops max, 60 byte packets
 1  172.26.31.129 (172.26.31.129)  0.000 ms  0.000 ms  0.003 ms
 2  * * *
 3  192.168.31.226 (192.168.31.226)  0.014 ms  0.005 ms  0.006 ms
root@Castelo2:/tmp/pycore.35235/Castelo2.conf# traceroute -I 192.168.31.234
traceroute to 192.168.31.234 (192.168.31.234), 30 hops max, 60 byte packets
 1  172.26.31.129 (172.26.31.129)  0.050 ms  0.009 ms  0.008 ms
 2  * * *
 3  192.168.31.234 (192.168.31.234)  0.148 ms  0.017 ms  0.014 ms
root@Castelo2:/tmp/pycore.35235/Castelo2.conf# 

```

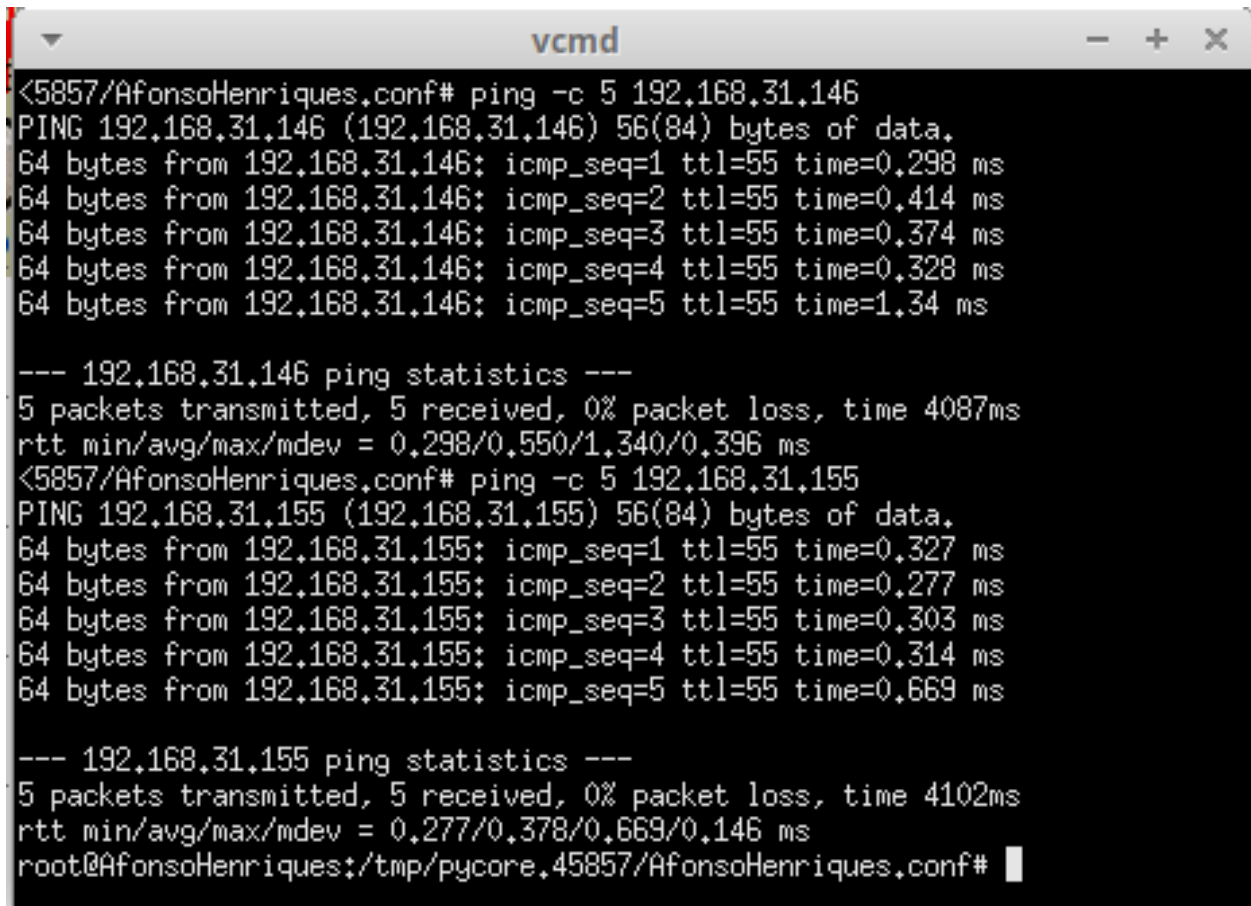
Figura 22: Tabela de encaminhamento resultante e testes de conectividade bem-sucedidos.

Utilidade de uma Rota Default: A rota default (representada por 0.0.0.0) funciona como um caminho de recurso. Serve para encaminhar pacotes destinados a qualquer rede que não esteja explicitamente listada na tabela de roteamento local. A sua grande utilidade reside na simplificação administrativa e na poupança de memória: em vez de os hosts terem de conhecer rotas exatas para todos os destinos possíveis, basta conhecerem as suas redes locais e enviarem o restante tráfego desconhecido para o *gateway* predefinido.

3.2. Questão 2

3.2.1. a)

Confirme, através do comando ping, que AfonsoHenriques tem efetivamente conectividade com os servidores Reddit e Twitch.



```
<5857/AfonsoHenriques.conf# ping -c 5 192.168.31.146
PING 192.168.31.146 (192.168.31.146) 56(84) bytes of data.
64 bytes from 192.168.31.146: icmp_seq=1 ttl=55 time=0.298 ms
64 bytes from 192.168.31.146: icmp_seq=2 ttl=55 time=0.414 ms
64 bytes from 192.168.31.146: icmp_seq=3 ttl=55 time=0.374 ms
64 bytes from 192.168.31.146: icmp_seq=4 ttl=55 time=0.328 ms
64 bytes from 192.168.31.146: icmp_seq=5 ttl=55 time=1.34 ms

--- 192.168.31.146 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4087ms
rtt min/avg/max/mdev = 0.298/0.550/1.340/0.396 ms
<5857/AfonsoHenriques.conf# ping -c 5 192.168.31.155
PING 192.168.31.155 (192.168.31.155) 56(84) bytes of data.
64 bytes from 192.168.31.155: icmp_seq=1 ttl=55 time=0.327 ms
64 bytes from 192.168.31.155: icmp_seq=2 ttl=55 time=0.277 ms
64 bytes from 192.168.31.155: icmp_seq=3 ttl=55 time=0.303 ms
64 bytes from 192.168.31.155: icmp_seq=4 ttl=55 time=0.314 ms
64 bytes from 192.168.31.155: icmp_seq=5 ttl=55 time=0.669 ms

--- 192.168.31.155 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4102ms
rtt min/avg/max/mdev = 0.277/0.378/0.669/0.146 ms
root@AfonsoHenriques:/tmp/pycore.45857/AfonsoHenriques.conf#
```

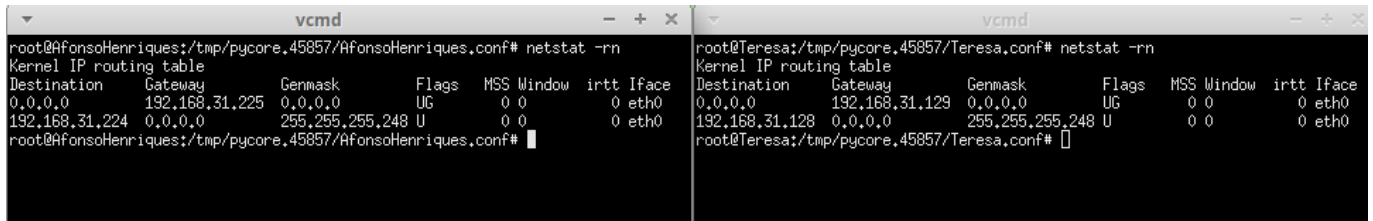
Figura 23: Sucesso no teste de ping para os servidores Twitch e Reddit a partir de AfonsoHenriques.

Para confirmar se o host **AfonsoHenriques** consegue comunicar com o exterior, enviámos pings para os servidores da Twitch (**192.168.31.146**) e do Reddit (**192.168.31.155**).

A Figura 23 mostra que não houve qualquer perda de pacotes (0% packet loss) em nenhum dos testes. Com isto, provamos que a ligação entre o Afonso, o seu router de acesso e a rede CDN está a funcionar sem problemas. Fica assim claro que a falha de comunicação com a Teresa não é culpa da rede local do Afonso, o que significa que o erro tem de estar escondido algures nos routers intermédios da infraestrutura.

3.2.2. b)

Recorrendo ao comando `netstat -rn`, analise as tabelas de encaminhamento dos dispositivos AfonsoHenriques e Teresa. Existe algum problema com as suas entradas? Identifique e descreva a utilidade de cada uma das entradas destes dois hosts.



Destination	Gateway	Genmask	Flags	MSS	Window	irtt	Iface
0.0.0.0	192.168.31.225	0.0.0.0	UG	0	0	0	eth0
192.168.31.224	0.0.0.0	255.255.255.248	U	0	0	0	eth0

Destination	Gateway	Genmask	Flags	MSS	Window	irtt	Iface
0.0.0.0	192.168.31.129	0.0.0.0	UG	0	0	0	eth0
192.168.31.128	0.0.0.0	255.255.255.248	U	0	0	0	eth0

Figura 24: Tabelas de encaminhamento local dos hosts AfonsoHenriques e Teresa.

Ao analisar as tabelas de encaminhamento dos hosts **AfonsoHenriques** e **Teresa** (Figura 24), observamos detalhadamente a utilidade de cada campo e respetivas entradas:

- **Destination (Destino):** Especifica a rede ou o endereço IP de destino.
 - ▶ `0.0.0.0` representa a rota *default* (por defeito), utilizada para encaminhar pacotes para qualquer destino que não tenha uma rota mais específica definida na tabela.
 - ▶ `192.168.31.224` e `192.168.31.128` correspondem às sub-redes locais de AfonsoHenriques e Teresa, respetivamente.
- **Gateway:** Indica o endereço IP do próximo salto (*next hop*).
 - ▶ O valor `0.0.0.0` assinala que a rede de destino está diretamente ligada à máquina, não sendo necessário atravessar nenhum router para a alcançar.
 - ▶ Os endereços `192.168.31.225` (no AfonsoHenriques) e `192.168.31.129` (na Teresa) representam os IPs das interfaces dos seus respetivos routers de acesso local (RACondado e RAGaliza), que fazem o encaminhamento de todo o tráfego para fora da sub-rede.
- **Genmask (Máscara de Rede):** Define a dimensão da rede de destino listada.
 - ▶ `255.255.255.248` corresponde à máscara da sub-rede local (/29), permitindo até 6 hosts utilizáveis.
 - ▶ `0.0.0.0` conjugado com o destino `0.0.0.0` indica que esta rota cobre todo e qualquer espaço de endereçamento IP.
- **Flags:** Indicam o estado e as características operacionais da rota.
 - ▶ **U** → A rota está ativa (*Up*).
 - ▶ **G** → A rota obriga à utilização de um *Gateway*.
 - ▶ Nas tabelas apresentadas, confirmamos a presença de rotas locais (U) e de rotas por defeito (UG).
- **MSS e Window:** Relacionados com parâmetros de controlo de ligações TCP. Encontram-se a `0`, pois o sistema operativo ajusta estes valores dinamicamente.
- **Iface (Interface de Rede):** Especifica a placa de rede por onde os pacotes vão sair. Em ambas as máquinas, todo o tráfego utiliza a `eth0`.

Conclusão: A análise comprova que **não existe qualquer problema** nas tabelas de encaminhamento dos dois hosts. Cada um possui uma configuração perfeitamente funcional e validada: uma rota direcionada à sua sub-rede local e uma rota *default* que aponta corretamente para o seu router. Deste modo, conclui-se que o problema de comunicação entre o Afonso e a Teresa não reside na configuração local, mas sim numa provável falha de roteamento nos routers intermédios (Core da rede).

3.2.3. c)

Analise o comportamento dos routers do core da rede (n1 a n6) quando tenta estabelecer comunicação entre os hosts AfonsoHenriques e Teresa. Indique que dispositivo(s) não permite(m) o encaminhamento correto dos pacotes. Seguidamente, avalie e explique a(s) causa(s) do funcionamento incorreto do dispositivo.

Utilize o comando `ip route add/del` para adicionar as rotas necessárias ou remover rotas incorretas. Verifique a sintaxe completa do comando a usar com `man ip-route` ou `man route`. Poderá também utilizar o comando `traceroute` para se certificar do caminho nó a nó. Considere a alínea resolvida assim que houver tráfego a chegar ao ISP CondaOnline.

Para resolver esta alínea, efetuou-se uma análise passo a passo do percurso dos pacotes na rede Core, acompanhando o progresso através de sucessivos testes de conectividade (`traceroute`) a partir de AfonsoHenriques para Teresa (192.168.31.130) e corrigindo as tabelas de rotas ao longo do caminho.

Passo 1: O Bloqueio no router n2

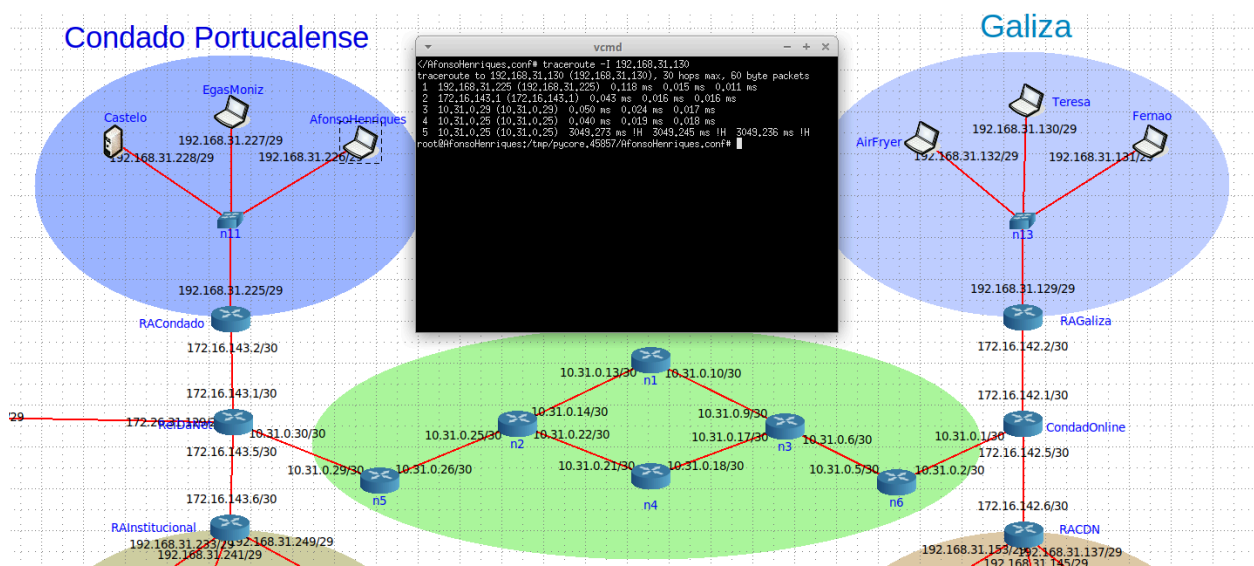


Figura 25: Traceroute inicial: falha de comunicação no router n2 (!H).

No primeiro teste (Figura 25), o pacote entrou corretamente no Core (via n5) mas esbarrou no router **n2** (10.31.0.25), que devolveu um erro de **Host Unreachable (!H)**.

```

vcmd
root@n2:/tmp/pycore.45857/n2.conf# netstat -rn
Kernel IP routing table
Destination      Gateway         Genmask         Flags         MSS Window  irtt Iface
10.31.0.0        10.31.0.13     255.255.255.252 UG            0 0        0 eth1
10.31.0.4        10.31.0.21     255.255.255.252 UG            0 0        0 eth0
10.31.0.8        10.31.0.13     255.255.255.252 UG            0 0        0 eth1
10.31.0.12       0.0.0.0        255.255.255.252 U            0 0        0 eth1
10.31.0.16       10.31.0.13     255.255.255.252 UG            0 0        0 eth1
10.31.0.20       0.0.0.0        255.255.255.252 U            0 0        0 eth0
10.31.0.24       0.0.0.0        255.255.255.252 U            0 0        0 eth2
10.31.0.28       10.31.0.26     255.255.255.252 UG            0 0        0 eth2
172.0.0.0        10.31.0.26     255.0.0.0       UG            0 0        0 eth2
172.16.142.0     10.31.0.13     255.255.255.252 UG            0 0        0 eth1
172.16.142.4     10.31.0.21     255.255.255.252 UG            0 0        0 eth0
172.16.143.0     10.31.0.26     255.255.255.252 UG            0 0        0 eth2
172.16.143.4     10.31.0.26     255.255.255.252 UG            0 0        0 eth2
192.168.31.128   10.31.0.13     255.255.255.248 UG            0 0        0 eth1
192.168.31.130   10.31.0.25     255.255.255.254 UG            0 0        0 eth2
192.168.31.136   10.31.0.21     255.255.255.248 UG            0 0        0 eth0
192.168.31.144   10.31.0.21     255.255.255.248 UG            0 0        0 eth0
192.168.31.152   10.31.0.21     255.255.255.248 UG            0 0        0 eth0
192.168.31.224   10.31.0.26     255.255.255.248 UG            0 0        0 eth2
192.168.31.232   10.31.0.26     255.255.255.248 UG            0 0        0 eth2
192.168.31.240   10.31.0.26     255.255.255.248 UG            0 0        0 eth2
192.168.31.248   10.31.0.26     255.255.255.248 UG            0 0        0 eth2
root@n2:/tmp/pycore.45857/n2.conf# 

```

Figura 26: Tabela de encaminhamento do router n2.

Ao consultar a tabela do **n2** (Figura 26), detetou-se a causa: existia uma rota específica para o IP da Teresa (192.168.31.130/31) a apontar para o próprio IP do **n2** (10.31.0.25). Sendo impossível um router enviar um pacote para si mesmo para alcançar outra rede, a solução passou por apagar esta rota enganadora, obrigando o router a usar a rota /29 genérica que apontava corretamente para o **n1**: `ip route del 192.168.31.130/31 via 10.31.0.25`

Passo 2: O Loop de Roteamento no n1

```
vcmd
</AfonsoHenriques.conf# traceroute -I 192.168.31.130
traceroute to 192.168.31.130 (192.168.31.130), 30 hops max, 60 byte packets
 1 192.168.31.225 (192.168.31.225)  0.099 ms  0.008 ms  0.006 ms
 2 172.16.143.1 (172.16.143.1)    0.044 ms  0.009 ms  0.008 ms
 3 10.31.0.29 (10.31.0.29)        0.049 ms  0.012 ms  0.016 ms
 4 10.31.0.25 (10.31.0.25)        0.046 ms  0.015 ms  0.015 ms
 5 10.31.0.13 (10.31.0.13)       0.069 ms  0.018 ms  0.019 ms
 6 10.31.0.25 (10.31.0.25)       0.019 ms  0.077 ms  0.019 ms
 7 10.31.0.13 (10.31.0.13)       0.023 ms  0.020 ms  0.021 ms
 8 * * *
 9 * * *
10 * * *
11 * * *
12 * * *
13 * 10.31.0.13 (10.31.0.13)    0.429 ms  0.984 ms
14 10.31.0.25 (10.31.0.25)      0.969 ms  0.966 ms  0.963 ms
15 10.31.0.13 (10.31.0.13)      0.993 ms  0.989 ms  0.986 ms
16 10.31.0.25 (10.31.0.25)      0.981 ms  0.976 ms *
```

Figura 27: Traceroute após correção no n2: loop infinito entre n1 e n2.

Repetindo o teste (Figura 27), descobriu-se um **routing loop**. O pacote andava aos saltos infinitamente entre o **n1** (10.31.0.13) e o **n2** (10.31.0.25).

```
vcmd
root@n1:/tmp/pycore.45857/n1.conf# netstat -rn
Kernel IP routing table
Destination    Gateway         Genmask         Flags   MSS Window  irtt  Iface
10.31.0.0      10.31.0.9       255.255.255.252 UG        0 0          0 eth0
10.31.0.4      10.31.0.9       255.255.255.252 UG        0 0          0 eth0
10.31.0.8      0.0.0.0         255.255.255.252 U         0 0          0 eth0
10.31.0.12     0.0.0.0         255.255.255.252 U         0 0          0 eth1
10.31.0.16     10.31.0.9       255.255.255.252 UG        0 0          0 eth0
10.31.0.20     10.31.0.14      255.255.255.252 UG        0 0          0 eth1
10.31.0.24     10.31.0.14      255.255.255.252 UG        0 0          0 eth1
10.31.0.28     10.31.0.14      255.255.255.252 UG        0 0          0 eth1
172.0.0.0      10.31.0.14      255.0.0.0       UG        0 0          0 eth1
172.16.142.0   10.31.0.9       255.255.255.252 UG        0 0          0 eth0
172.16.142.4   10.31.0.9       255.255.255.252 UG        0 0          0 eth0
172.16.143.0   10.31.0.14      255.255.255.252 UG        0 0          0 eth1
172.16.143.4   10.31.0.14      255.255.255.252 UG        0 0          0 eth1
192.168.31.128 10.31.0.14      255.255.255.248 UG        0 0          0 eth1
192.168.31.136 10.31.0.9       255.255.255.248 UG        0 0          0 eth0
192.168.31.144 10.31.0.9       255.255.255.248 UG        0 0          0 eth0
192.168.31.152 10.31.0.9       255.255.255.248 UG        0 0          0 eth0
192.168.31.224 10.31.0.14      255.255.255.248 UG        0 0          0 eth1
192.168.31.232 10.31.0.14      255.255.255.248 UG        0 0          0 eth1
192.168.31.240 10.31.0.14      255.255.255.248 UG        0 0          0 eth1
192.168.31.248 10.31.0.14      255.255.255.248 UG        0 0          0 eth1
```

Figura 28: Tabela de encaminhamento do router n1.

A verificação da tabela do **n1** (Figura 28) explicou o sucedido: a rota para a sub-rede da Teresa (192.168.31.128) apontava de volta para o IP do **n2** (10.31.0.14). Para quebrar o ciclo e fazer o pacote avançar em direção à Galiza, removeu-se a rota errada e adicionou-se uma nova apontando para a interface correta do router **n3** (10.31.0.9):

```
ip route del 192.168.31.128/29 via 10.31.0.14
ip route add 192.168.31.128/29 via 10.31.0.9
```

Passo 3: A Rota em Falta no n3

```
vcmd
</AfonsoHenriques.conf# traceroute -I 192.168.31.130
traceroute to 192.168.31.130 (192.168.31.130), 30 hops max, 60 byte packets
 1 192.168.31.225 (192.168.31.225) 0.089 ms 0.011 ms 0.010 ms
 2 172.16.143.1 (172.16.143.1) 0.036 ms 0.013 ms 0.011 ms
 3 10.31.0.29 (10.31.0.29) 0.039 ms 0.016 ms 0.016 ms
 4 10.31.0.25 (10.31.0.25) 0.048 ms 0.019 ms 0.020 ms
 5 10.31.0.13 (10.31.0.13) 0.048 ms 0.023 ms 0.023 ms
 6 10.31.0.17 (10.31.0.17) 0.113 ms !N 0.109 ms !N *
root@AfonsoHenriques:/tmp/pycore.45857/AfonsoHenriques.conf#
```

Figura 29: Traceroute após correção no n1: paragem no n3 com erro !N.

Livre do loop, o pacote conseguiu alcançar o router **n3** (Figura 29), mas foi descartado com um erro de **Network Unreachable (!N)**.

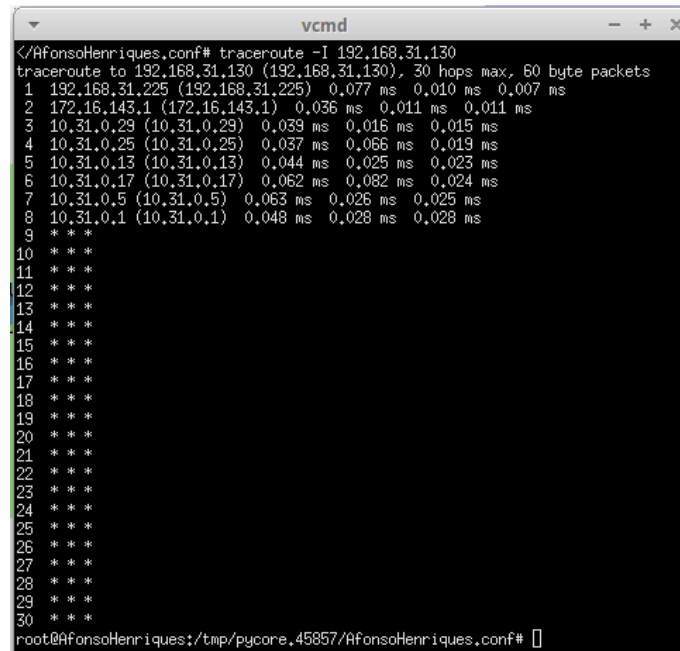
```
vcmd
root@n3:/tmp/pycore.45857/n3.conf# netstat -rn
Kernel IP routing table
Destination      Gateway         Genmask         Flags   MSS Window  irtt  Iface
10.31.0.0        10.31.0.5      255.255.255.252 UG      0 0        0     eth2
10.31.0.4        0.0.0.0        255.255.255.252 U        0 0        0     eth2
10.31.0.8        0.0.0.0        255.255.255.252 U        0 0        0     eth0
10.31.0.12       10.31.0.10     255.255.255.252 UG      0 0        0     eth0
10.31.0.16       0.0.0.0        255.255.255.252 U        0 0        0     eth1
10.31.0.20       10.31.0.18     255.255.255.252 UG      0 0        0     eth1
10.31.0.24       10.31.0.18     255.255.255.252 UG      0 0        0     eth1
10.31.0.28       10.31.0.10     255.255.255.252 UG      0 0        0     eth0
172.0.0.0        10.31.0.10     255.0.0.0      UG      0 0        0     eth0
172.16.142.0     10.31.0.5      255.255.255.252 UG      0 0        0     eth2
172.16.142.4     10.31.0.5      255.255.255.252 UG      0 0        0     eth2
172.16.143.0     10.31.0.18     255.255.255.252 UG      0 0        0     eth1
172.16.143.4     10.31.0.10     255.255.255.252 UG      0 0        0     eth0
192.168.31.136   10.31.0.5      255.255.255.248 UG      0 0        0     eth2
192.168.31.144   10.31.0.5      255.255.255.248 UG      0 0        0     eth2
192.168.31.152   10.31.0.5      255.255.255.248 UG      0 0        0     eth2
192.168.31.224   10.31.0.18     255.255.255.248 UG      0 0        0     eth1
192.168.31.232   10.31.0.10     255.255.255.248 UG      0 0        0     eth0
192.168.31.240   10.31.0.10     255.255.255.248 UG      0 0        0     eth0
192.168.31.248   10.31.0.10     255.255.255.248 UG      0 0        0     eth0
root@n3:/tmp/pycore.45857/n3.conf#
```

Figura 30: Tabela de encaminhamento do router n3.

Como evidenciado na tabela do **n3** (Figura 30), não existia qualquer entrada para a sub-rede de destino (192.168.31.128/29). O router simplesmente não sabia o que fazer ao pacote. O problema foi sanado adicionando a rota em falta, orientando o tráfego para a interface do router seguinte no Core, o **n6** (10.31.0.5):

```
ip route add 192.168.31.128/29 via 10.31.0.5
```

Conclusão e Verificação Final



```
vcmd
</AfonsoHenriques.conf# traceroute -I 192.168.31.130
traceroute to 192.168.31.130 (192.168.31.130), 30 hops max, 60 byte packets
 1 192.168.31.225 (192.168.31.225) 0.077 ms 0.010 ms 0.007 ms
 2 172.16.143.1 (172.16.143.1) 0.036 ms 0.011 ms 0.011 ms
 3 10.31.0.29 (10.31.0.29) 0.039 ms 0.016 ms 0.015 ms
 4 10.31.0.25 (10.31.0.25) 0.037 ms 0.066 ms 0.019 ms
 5 10.31.0.13 (10.31.0.13) 0.044 ms 0.025 ms 0.023 ms
 6 10.31.0.17 (10.31.0.17) 0.062 ms 0.082 ms 0.024 ms
 7 10.31.0.5 (10.31.0.5) 0.063 ms 0.026 ms 0.025 ms
 8 10.31.0.1 (10.31.0.1) 0.048 ms 0.028 ms 0.028 ms
 9 * * *
10 * * *
11 * * *
12 * * *
13 * * *
14 * * *
15 * * *
16 * * *
17 * * *
18 * * *
19 * * *
20 * * *
21 * * *
22 * * *
23 * * *
24 * * *
25 * * *
26 * * *
27 * * *
28 * * *
29 * * *
30 * * *
root@AfonsoHenriques:/tmp/pycore.45857/AfonsoHenriques.conf#
```

Figura 31: Traceroute final indicando o alcance do ISP CondadOnline.

Com as três correções efetuadas, lançou-se um derradeiro traceroute (Figura 31). Como se pode constatar no salto número 8, o tráfego atingiu com sucesso o IP 10.31.0.1. Sendo este o endereço da interface de entrada do ISP **CondadOnline**, dá-se a condição proposta no enunciado como resolvida.

3.2.4. d)

Uma vez que o core da rede esteja a encaminhar corretamente os pacotes enviados por AfonsoHenriques, confira com o Wireshark se estes são recebidos por Teresa.

3.2.4.1. i)

Em caso afirmativo, porque é que continua a não existir conectividade entre D.Teresa e D.Afonso Henriques? Efetue as alterações necessárias para garantir que a conectividade é restabelecida e o confronto entre os dois é evitado.

Passo 1: Verificação de recepção no Wireshark

No.	Time	Source	Destination	Protocol	Length	Info
8	12.014661704	192.168.31.193	224.0.0.5	OSPF	78	Hello Packet
9	14.015411451	192.168.31.193	224.0.0.5	OSPF	78	Hello Packet
10	16.016184432	192.168.31.193	224.0.0.5	OSPF	78	Hello Packet
11	17.796127303	fe80::200:ff:feaa:10	ff02::5	OSPF	90	Hello Packet
12	18.016693589	192.168.31.193	224.0.0.5	OSPF	78	Hello Packet
13	20.017244847	192.168.31.193	224.0.0.5	OSPF	78	Hello Packet
14	22.017682860	192.168.31.193	224.0.0.5	OSPF	78	Hello Packet
15	24.018479586	192.168.31.193	224.0.0.5	OSPF	78	Hello Packet
16	26.018695202	192.168.31.193	224.0.0.5	OSPF	78	Hello Packet
17	27.767706866	fe80::200:ff:feaa:10	ff02::5	OSPF	90	Hello Packet
18	28.019691287	192.168.31.193	224.0.0.5	OSPF	78	Hello Packet
19	30.020451626	192.168.31.193	224.0.0.5	OSPF	78	Hello Packet
20	32.020757366	192.168.31.193	224.0.0.5	OSPF	78	Hello Packet
21	34.021606934	192.168.31.193	224.0.0.5	OSPF	78	Hello Packet
22	36.021979679	192.168.31.193	224.0.0.5	OSPF	78	Hello Packet
23	36.816391260	192.168.31.226	192.168.31.130	ICMP	98	Echo (ping) request id=0x0042, seq=1/256, ttl=55 (reply in 24)
24	36.816422622	192.168.31.130	192.168.31.226	ICMP	98	Echo (ping) reply id=0x0042, seq=1/256, ttl=64 (request in 23)
25	36.816439467	192.168.31.129	192.168.31.130	ICMP	126	Destination unreachable (Network unreachable)
26	37.819300879	fe80::200:ff:feaa:10	ff02::5	OSPF	90	Hello Packet
27	37.843884024	192.168.31.226	192.168.31.130	ICMP	98	Echo (ping) request id=0x0042, seq=2/512, ttl=55 (reply in 28)
28	37.843920864	192.168.31.130	192.168.31.226	ICMP	98	Echo (ping) reply id=0x0042, seq=2/512, ttl=64 (request in 27)
29	37.843943949	192.168.31.129	192.168.31.130	ICMP	126	Destination unreachable (Network unreachable)
30	38.023146971	192.168.31.193	224.0.0.5	OSPF	78	Hello Packet
31	38.867142855	192.168.31.226	192.168.31.130	ICMP	98	Echo (ping) request id=0x0042, seq=3/768, ttl=55 (reply in 32)
32	38.867226669	192.168.31.130	192.168.31.226	ICMP	98	Echo (ping) reply id=0x0042, seq=3/768, ttl=64 (request in 31)
33	38.867260261	192.168.31.129	192.168.31.130	ICMP	126	Destination unreachable (Network unreachable)
34	39.890777675	192.168.31.226	192.168.31.130	ICMP	98	Echo (ping) request id=0x0042, seq=4/1024, ttl=55 (reply in 35)
35	39.890809929	192.168.31.130	192.168.31.226	ICMP	98	Echo (ping) reply id=0x0042, seq=4/1024, ttl=64 (request in 34)
36	39.890836645	192.168.31.129	192.168.31.130	ICMP	126	Destination unreachable (Network unreachable)
37	40.023668720	192.168.31.193	224.0.0.5	OSPF	78	Hello Packet
38	42.023909603	192.168.31.193	224.0.0.5	OSPF	78	Hello Packet
39	42.034630899	00:00:00_aa:00:13	00:00:00_aa:00:10	ARP	42	Who has 192.168.31.129? Tell 192.168.31.130
40	42.034791042	00:00:00_aa:00:10	00:00:00_aa:00:13	ARP	42	Who has 192.168.31.130? Tell 192.168.31.129
41	42.034919901	00:00:00_aa:00:10	00:00:00_aa:00:13	ARP	42	192.168.31.129 is at 00:00:00_aa:00:10
42	42.034858792	00:00:00_aa:00:13	00:00:00_aa:00:10	ARP	42	192.168.31.130 is at 00:00:00_aa:00:13
43	44.024728493	192.168.31.193	224.0.0.5	OSPF	78	Hello Packet

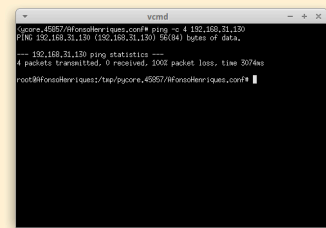
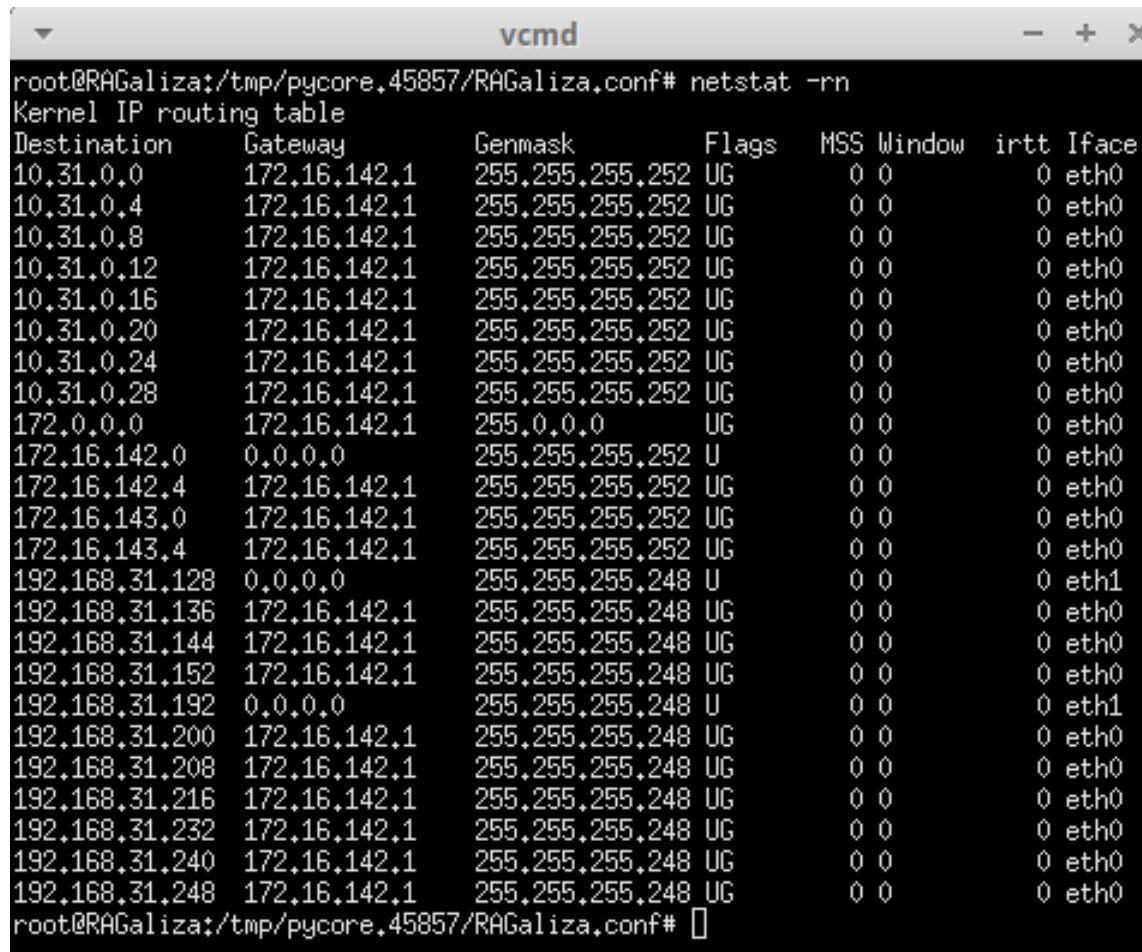


Figura 32: Captura de tráfego no Wireshark da interface da Teresa.

Como o caminho de ida pelo Core foi corrigido na alínea anterior, o tráfego de **AfonsoHenriques** (192.168.31.226) já consegue alcançar a **Teresa** (192.168.31.130). A captura efetuada no Wireshark (Figura 32) prova exatamente isso: na linha 23, regista-se a chegada do **Echo (ping) request** ao destino.

No entanto, a conectividade bidirecional continua a falhar. Na linha 24, a Teresa gera a devida resposta (**Echo ping reply**), mas imediatamente a seguir (linha 25), o seu router de acesso local (**RAGaliza**, com o IP 192.168.31.129) interceta a resposta e descarta-a, devolvendo à máquina o erro de **Destination unreachable (Network unreachable)**. Isto indica inequivocamente que o problema de comunicação reside no trajeto de regresso.

Passo 2: Diagnóstico do Caminho de Regresso



```
root@RAGaliza:/tmp/pycore.45857/RAGaliza.conf# netstat -rn
Kernel IP routing table
Destination      Gateway         Genmask         Flags   MSS Window  irtt  Iface
10.31.0.0        172.16.142.1   255.255.255.252 UG      0  0        0     eth0
10.31.0.4        172.16.142.1   255.255.255.252 UG      0  0        0     eth0
10.31.0.8        172.16.142.1   255.255.255.252 UG      0  0        0     eth0
10.31.0.12       172.16.142.1   255.255.255.252 UG      0  0        0     eth0
10.31.0.16       172.16.142.1   255.255.255.252 UG      0  0        0     eth0
10.31.0.20       172.16.142.1   255.255.255.252 UG      0  0        0     eth0
10.31.0.24       172.16.142.1   255.255.255.252 UG      0  0        0     eth0
10.31.0.28       172.16.142.1   255.255.255.252 UG      0  0        0     eth0
172.0.0.0        172.16.142.1   255.0.0.0       UG      0  0        0     eth0
172.16.142.0     0.0.0.0        255.255.255.252 U        0  0        0     eth0
172.16.142.4     172.16.142.1   255.255.255.252 UG      0  0        0     eth0
172.16.143.0     172.16.142.1   255.255.255.252 UG      0  0        0     eth0
172.16.143.4     172.16.142.1   255.255.255.252 UG      0  0        0     eth0
192.168.31.128   0.0.0.0        255.255.255.248 U        0  0        0     eth1
192.168.31.136   172.16.142.1   255.255.255.248 UG      0  0        0     eth0
192.168.31.144   172.16.142.1   255.255.255.248 UG      0  0        0     eth0
192.168.31.152   172.16.142.1   255.255.255.248 UG      0  0        0     eth0
192.168.31.192   0.0.0.0        255.255.255.248 U        0  0        0     eth1
192.168.31.200   172.16.142.1   255.255.255.248 UG      0  0        0     eth0
192.168.31.208   172.16.142.1   255.255.255.248 UG      0  0        0     eth0
192.168.31.216   172.16.142.1   255.255.255.248 UG      0  0        0     eth0
192.168.31.232   172.16.142.1   255.255.255.248 UG      0  0        0     eth0
192.168.31.240   172.16.142.1   255.255.255.248 UG      0  0        0     eth0
192.168.31.248   172.16.142.1   255.255.255.248 UG      0  0        0     eth0
root@RAGaliza:/tmp/pycore.45857/RAGaliza.conf#
```

Figura 33: Tabela de encaminhamento do router RAGaliza, evidenciando a falta de rotas.

A verificação da tabela de rotas do **RAGaliza** (Figura 33) confirmou a suspeita levantada pelo erro ICMP. O equipamento possui conhecimento de várias sub-redes, mas **não tem qualquer entrada configurada para a sub-rede do Condado Portucalense** (192.168.31.224/29). Devido a esta omissão, o router não sabe por onde deve expedir a resposta destinada ao Afonso, optando pelo seu descarte.

Passo 3: Restabelecimento da Conectividade



The image shows two terminal windows side-by-side. The left window, titled 'vcmd', shows a ping command being executed from a device named 'core,45857/AfonsoHenriques.conf#'. The command is 'ping -c 4 192.168.31.130'. The output shows four successful ping attempts with varying times (0.288 ms, 0.585 ms, 0.441 ms, 0.519 ms). Below the ping results, a summary line states: '--- 192.168.31.130 ping statistics --- 4 packets transmitted, 4 received, 0% packet loss, time 3077ms rtt min/avg/max/mdev = 0.288/0.458/0.585/0.110 ms'. The right window, also titled 'vcmd', shows a configuration command being entered on a device named '5857/RAGaliza.conf#'. The command is 'ip route add 192.168.31.224/29 via 172.16.142.1'. The prompt changes to 'root@RAGaliza:/tmp/pycore,45857/RAGaliza.conf#'.

```
<core,45857/AfonsoHenriques.conf# ping -c 4 192.168.31.130
PING 192.168.31.130 (192.168.31.130) 56(84) bytes of data:
64 bytes from 192.168.31.130: icmp_seq=1 ttl=55 time=0.288 ms
64 bytes from 192.168.31.130: icmp_seq=2 ttl=55 time=0.585 ms
64 bytes from 192.168.31.130: icmp_seq=3 ttl=55 time=0.441 ms
64 bytes from 192.168.31.130: icmp_seq=4 ttl=55 time=0.519 ms

--- 192.168.31.130 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3077ms
rtt min/avg/max/mdev = 0.288/0.458/0.585/0.110 ms
root@AfonsoHenriques:/tmp/pycore,45857/AfonsoHenriques.conf#
```

```
<5857/RAGaliza.conf# ip route add 192.168.31.224/29 via 172.16.142.1
root@RAGaliza:/tmp/pycore,45857/RAGaliza.conf#
```

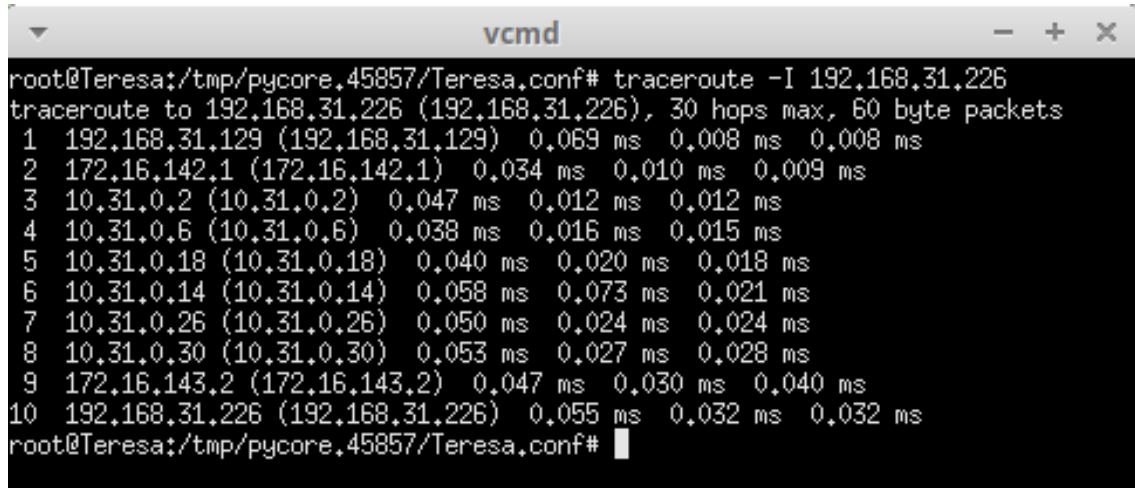
Figura 34: Adição da rota em falta no RAGaliza e consequente sucesso no teste de ping.

Para sanar a anomalia e restabelecer a comunicação, foi imperativo ensinar ao **RAGaliza** que o tráfego destinado à rede do Condado deve ser encaminhado para o router do ISP vizinho (IP 172.16.142.1). A correção foi aplicada com recurso ao seguinte comando: `ip route add 192.168.31.224/29 via 172.16.142.1`

Subsequentemente à inserção da rota, efetuou-se um novo teste de conectividade a partir do **AfonsoHenriques**. Conforme ilustrado na Figura 34, o comando `ping` apresentou agora 100% de eficácia (0% packet loss). Fica assim provado que quer a rota de ida pelo Core, quer a rota de regresso pelas infraestruturas dos ISPs se encontram plenamente operacionais, evitando-se o confronto relatado no enunciado.

3.2.4.2. ii)

As rotas dos pacotes ICMP echo reply são as mesmas, mas em sentido inverso, que as rotas dos pacotes ICMP echo request enviados entre AfonsoHenriques e Teresa? (Sugestão: analise as rotas nos dois sentidos com o traceroute). Mostre graficamente a rota seguida nos dois sentidos por esses pacotes ICMP.



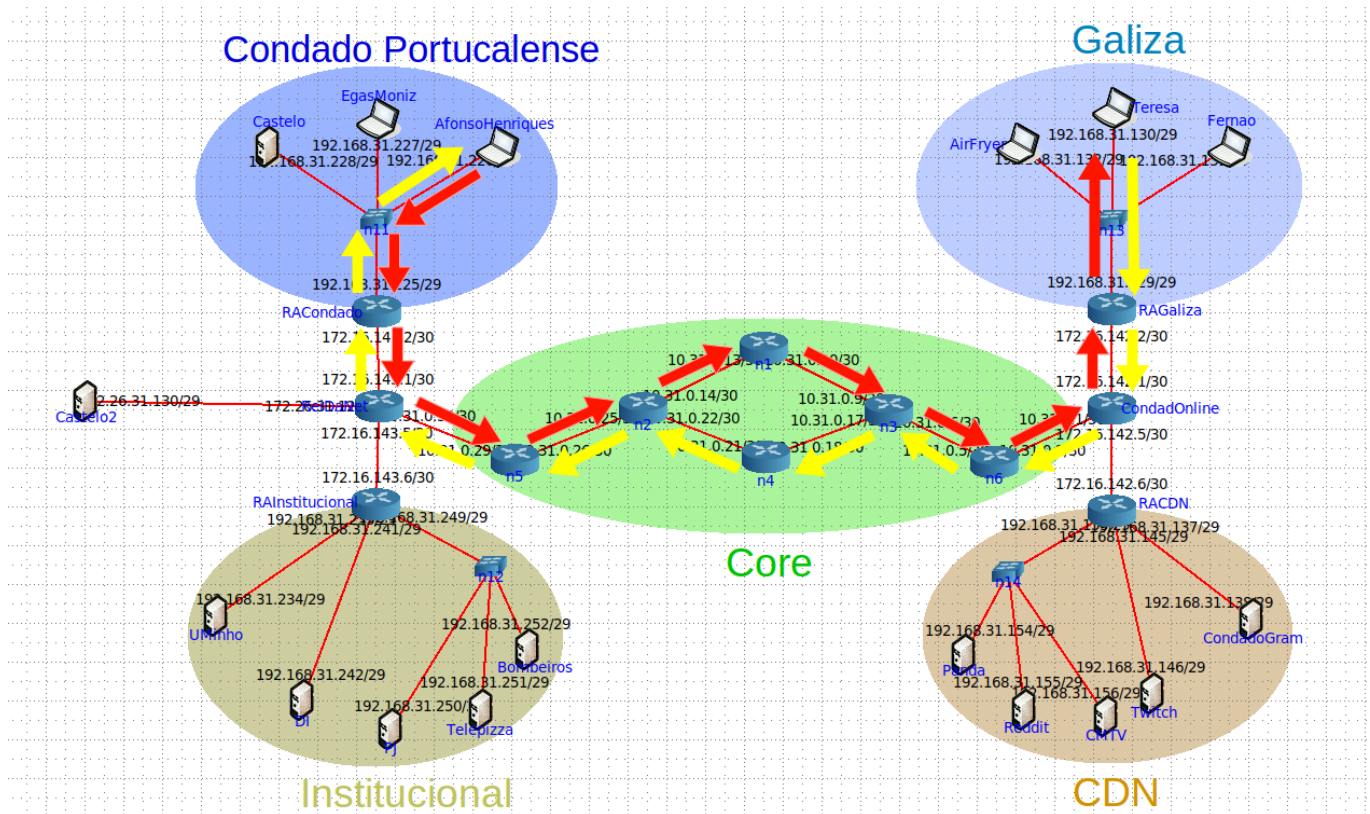
```
vcmd
root@Teresa:/tmp/pycore.45857/Teresa.conf# traceroute -I 192.168.31.226
traceroute to 192.168.31.226 (192.168.31.226), 30 hops max, 60 byte packets
 1 192.168.31.129 (192.168.31.129) 0.069 ms 0.008 ms 0.008 ms
 2 172.16.142.1 (172.16.142.1) 0.034 ms 0.010 ms 0.009 ms
 3 10.31.0.2 (10.31.0.2) 0.047 ms 0.012 ms 0.012 ms
 4 10.31.0.6 (10.31.0.6) 0.038 ms 0.016 ms 0.015 ms
 5 10.31.0.18 (10.31.0.18) 0.040 ms 0.020 ms 0.018 ms
 6 10.31.0.14 (10.31.0.14) 0.058 ms 0.073 ms 0.021 ms
 7 10.31.0.26 (10.31.0.26) 0.050 ms 0.024 ms 0.024 ms
 8 10.31.0.30 (10.31.0.30) 0.053 ms 0.027 ms 0.028 ms
 9 172.16.143.2 (172.16.143.2) 0.047 ms 0.030 ms 0.040 ms
10 192.168.31.226 (192.168.31.226) 0.055 ms 0.032 ms 0.032 ms
root@Teresa:/tmp/pycore.45857/Teresa.conf#
```

Figura 35: Traceroute no sentido inverso (Teresa para AfonsoHenriques).

Após a reposição da conectividade total, procedeu-se à análise comparativa dos percursos de ida e volta utilizando o comando `traceroute` nas duas extremidades (Figura 35).

Conclusão: As rotas não são simétricas. Embora os pontos de origem e destino sejam os mesmos, o caminho percorrido pelos pacotes no sentido inverso (Teresa → Afonso) difere do caminho de ida (Afonso → Teresa). Este fenómeno é conhecido em redes como **Assimetria de Roteamento** (*Asymmetric Routing*).

A imagem seguinte ilustra graficamente a discrepância entre os dois trajetos:



A divergência ocorre estritamente no meio da malha da rede Core (saltos 5 e 6). No trajeto de ida, o tráfego é escoado através do router **n1**, enquanto no trajeto de regresso os pacotes seguem pelo router **n4**.

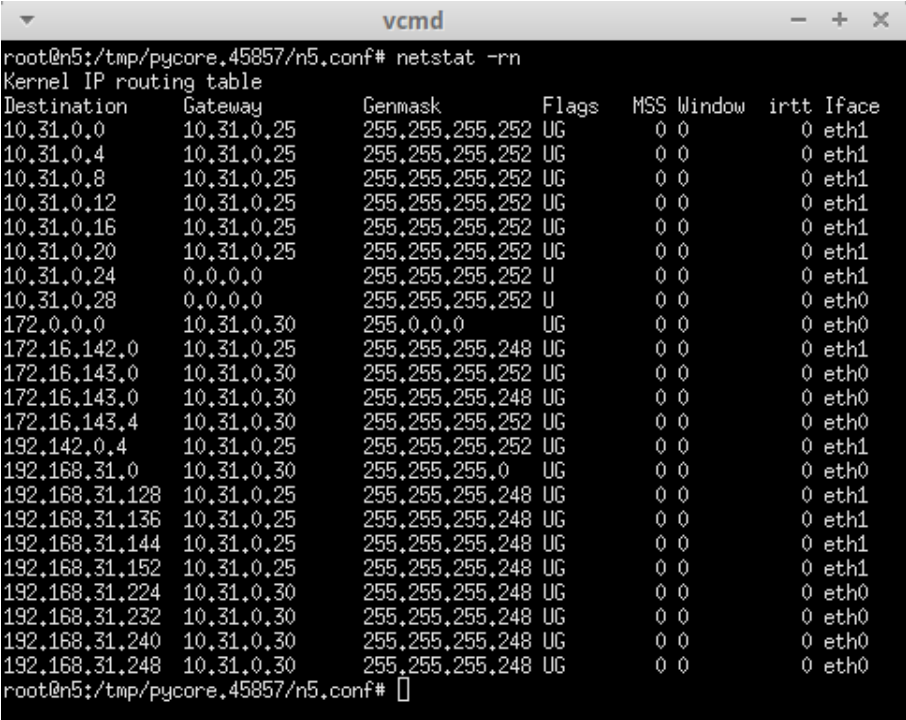
A causa desta assimetria reside no funcionamento do protocolo de encaminhamento dinâmico **OSPF**. Cada router toma decisões de roteamento de forma independente, calculando o melhor caminho (menor custo) para o destino a partir da sua própria perspetiva na topologia. Devido às métricas e custos associados às ligações no Core verde, o custo acumulado no caminho via **n4** revelou-se mais vantajoso no sentido Galiza-Condado do que o caminho via **n1**.

3.2.5. e)

Estando restabelecida a conectividade entre os dois hosts, obtenha a tabela de encaminhamento de n5 e foque-se na seguinte entrada:

```
ip route 192.168.31.0 255.255.255.0 10.31.0.30
```

Existe uma correspondência (match) nesta entrada para pacotes enviados para o polo Galiza? E para CDN? Caso seja essa a entrada utilizada para o encaminhamento, permitirá o funcionamento esperado do dispositivo? Ofereça uma explicação pela qual essa entrada é ou não utilizada.



Destination	Gateway	Genmask	Flags	MSS	Window	irtt	Iface
10.31.0.0	10.31.0.25	255.255.255.252	UG	0	0	0	eth1
10.31.0.4	10.31.0.25	255.255.255.252	UG	0	0	0	eth1
10.31.0.8	10.31.0.25	255.255.255.252	UG	0	0	0	eth1
10.31.0.12	10.31.0.25	255.255.255.252	UG	0	0	0	eth1
10.31.0.16	10.31.0.25	255.255.255.252	UG	0	0	0	eth1
10.31.0.20	10.31.0.25	255.255.255.252	UG	0	0	0	eth1
10.31.0.24	0.0.0.0	255.255.255.252	U	0	0	0	eth1
10.31.0.28	0.0.0.0	255.255.255.252	U	0	0	0	eth0
172.0.0.0	10.31.0.30	255.0.0.0	UG	0	0	0	eth0
172.16.142.0	10.31.0.25	255.255.255.248	UG	0	0	0	eth1
172.16.143.0	10.31.0.30	255.255.255.252	UG	0	0	0	eth0
172.16.143.0	10.31.0.30	255.255.255.248	UG	0	0	0	eth0
172.16.143.4	10.31.0.30	255.255.255.252	UG	0	0	0	eth0
192.142.0.4	10.31.0.25	255.255.255.252	UG	0	0	0	eth1
192.168.31.0	10.31.0.30	255.255.255.0	UG	0	0	0	eth0
192.168.31.128	10.31.0.25	255.255.255.248	UG	0	0	0	eth1
192.168.31.136	10.31.0.25	255.255.255.248	UG	0	0	0	eth1
192.168.31.144	10.31.0.25	255.255.255.248	UG	0	0	0	eth1
192.168.31.152	10.31.0.25	255.255.255.248	UG	0	0	0	eth1
192.168.31.224	10.31.0.30	255.255.255.248	UG	0	0	0	eth0
192.168.31.232	10.31.0.30	255.255.255.248	UG	0	0	0	eth0
192.168.31.240	10.31.0.30	255.255.255.248	UG	0	0	0	eth0
192.168.31.248	10.31.0.30	255.255.255.248	UG	0	0	0	eth0

Figura 37: Tabela de encaminhamento do router n5 com destaque para as rotas analisadas.

Ao analisarmos a tabela de encaminhamento do router **n5** (Figura 37), verificamos a existência da entrada genérica para a rede 192.168.31.0/24 com o gateway 10.31.0.30. Respondendo diretamente às questões formuladas:

1. Existe correspondência (match) para o polo Galiza e para a CDN?

- **Sim, existe.** A máscara 255.255.255.0 (/24) abrange todos os endereços IP desde 192.168.31.0 até 192.168.31.255.
- Como a sub-rede da Galiza é a 192.168.31.128/29 e a sub-rede da CDN é a 192.168.31.136/29, ambas encontram-se totalmente incluídas neste intervalo mais vasto. Do ponto de vista puramente matemático, os pacotes para estes polos dão *match* com esta entrada.

2. Caso seja utilizada, permitirá o funcionamento esperado?

- **Não.** Se esta entrada fosse efetivamente utilizada, o tráfego seria perdido.
- O gateway estipulado nesta rota é o 10.31.0.30, que corresponde à interface do router **ReiDaNet**. Isto significa que os pacotes seriam enviados na direção do polo do Condado

Portugalense, afastando-se do seu destino real (que exige o encaminhamento em direção à rede Core, para a “direita”).

3. Explicação pela qual essa entrada não é utilizada: A entrada não é utilizada para estes destinos em virtude do mecanismo central de decisão de encaminhamento IP: o princípio do **Longest Prefix Match** (Correspondência do Prefixo Mais Longo).

Na tabela do router **n5**, para além da referida rota abrangente /24, existem rotas muito mais específicas e detalhadas:

- Uma rota para a Galiza (192.168.31.128) com a máscara 255.255.255.248 (/29).
- Uma rota para a CDN (192.168.31.136) com a máscara 255.255.255.248 (/29).

Perante um pacote, o router compara o endereço de destino com as rotas disponíveis e escolhe sempre a que tem a máscara maior (ou seja, o prefixo mais específico). Como /29 é mais específico que /24, o router **n5** ignora a rota genérica e utiliza as rotas /29, que possuem o gateway correto (10.31.0.25), garantindo a entrega dos pacotes ao router **n2** da rede Core.

3.2.6. f)

Os endereços utilizados pelos quatro polos são endereços públicos ou privados? E os utilizados no core da rede/ISPs? Justifique convenientemente.

Todos os endereços utilizados, tanto nos quatro polos como no Core da rede e nos ISPs, são endereços privados.

Segundo a norma **RFC 1918**, foram reservadas gamas específicas de endereços IP para utilização exclusiva em redes privadas (Intranets), as quais não são roteáveis na Internet pública. No cenário analisado, identificamos as seguintes gamas:

1. **Nos Polos:** É utilizada a gama 192.168.31.0/24. Esta pertence ao bloco de Classe C reservado para uso privado (192.168.0.0 a 192.168.255.255).
2. **No Core da Rede:** São utilizados endereços na gama 10.31.0.0/x. Estes pertencem ao bloco de Classe A reservado para uso privado (10.0.0.0 a 10.255.255.255).
3. **Nos ISPs e interligações:** São utilizados endereços na gama 172.16.142.0/x. Estes pertencem ao bloco de Classe B reservado para uso privado (172.16.0.0 a 172.31.255.255).

A utilização destes endereços é tecnicamente justificada pela necessidade de preservar o espaço limitado de endereçamento IPv4 público. Como esta infraestrutura representa uma rede de simulação laboratorial fechada, o uso de endereços privados permite criar uma topologia complexa sem conflitos com a rede externa, garantindo que toda a comunicação entre o Afonso e a Teresa ocorra dentro de um ambiente controlado e isolado da Internet global.

3.2.7. g)

Os switches localizados em cada um dos polos têm um endereço IP atribuído? Porquê?

No contexto do encaminhamento de dados entre os hosts, os switches localizados nos polos não têm (nem necessitam de) um endereço IP atribuído.

Esta ausência de IP justifica-se pelo nível em que estes dispositivos operam no modelo de referência OSI:

1. **Operação em Camada 2 (Data Link):** Os switches são dispositivos de Camada 2, o que significa que o seu funcionamento baseia-se em endereços físicos (MAC Addresses) e não em endereços lógicos (IP). Eles utilizam uma tabela de comutação (*MAC Address Table*) para encaminhar tramas de uma porta para outra, sendo “transparentes” ao protocolo IP que circula acima deles.
2. **Encaminhamento vs. Gestão:** Para que o tráfego do Afonso chegue ao router e vice-versa, o switch apenas precisa de ler os cabeçalhos Ethernet. Um endereço IP num switch serve apenas para fins de gestão e administração (permitindo que um técnico aceda ao dispositivo via rede para monitorização ou configuração), não tendo qualquer influência no transporte dos pacotes entre os utilizadores da rede local.

Desta forma, na topologia simulada, os switches atuam como elementos de comutação pura, ligando os hosts aos respetivos gateways sem necessidade de identificação na camada de rede.

3.3. Questão 3

Ao ver as fotos no CondadoGram, D. Teresa não ficou convencida com as novas alterações e ordena que Afonso Henriques vá arrumar o castelo. Inconformado, este decide planejar um novo ataque, mas constata que o seu exército não só perde bastante tempo a decidir que direção tomar a cada salto como, por vezes, inclusivamente se perde. De modo a facilitar a travessia, elimine as rotas referentes a Galiza e CDN no dispositivo n6 e defina um esquema de sumarização de rotas (Supernetting) que permita o uso de apenas uma rota para ambos os polos. Confirme que a conectividade é mantida e chame o docente para explicar o processo efetuado.

O objetivo deste exercício consistiu na otimização da tabela de encaminhamento do router **n6** através do processo de sumarização de rotas (**Supernetting**). O router n6 atua como o último salto do Core antes de entregar o tráfego ao ISP CondadOnline, possuindo inicialmente múltiplas rotas detalhadas para as sub-redes de destino.

Análise do Endereçamento e Cálculo da Supernet: Constatou-se que os polos da Galiza e da CDN eram compostos por quatro sub-redes de máscara /29:

- **Galiza:** 192.168.31.128/29
- **CDN (Sub-rede 1):** 192.168.31.136/29
- **CDN (Sub-rede 2):** 192.168.31.144/29
- **CDN (Sub-rede 3):** 192.168.31.152/29

Este bloco cobre o intervalo contínuo de endereços desde o .128 até ao .159, totalizando 32 endereços IP. Para englobar 32 endereços, são necessários 5 bits dedicados aos hosts ($2^5 = 32$). Consequentemente, a porção de rede passará a ocupar 27 bits ($32 - 5 = 27$). Deste modo, foi possível calcular uma única rota sumarizada de máscara /27 (Genmask 255.255.255.224): **192.168.31.128/27**.

Execução no Router n6: No terminal do dispositivo **n6**, procedeu-se à limpeza da tabela, eliminando as quatro rotas individuais com o comando `ip route del`. Em seguida, adicionou-se a rota agregada apontando para a interface de entrada do router CondadOnline (10.31.0.1) através do comando:

```
ip route add 192.168.31.128/27 via 10.31.0.1
```

```

root@n6:/tmp/pycore.39263/n6.conf# ip route del 192.168.31.128/29
root@n6:/tmp/pycore.39263/n6.conf# ip route del 192.168.31.136/29
root@n6:/tmp/pycore.39263/n6.conf# ip route del 192.168.31.144/29
root@n6:/tmp/pycore.39263/n6.conf# ip route del 192.168.31.152/29
root@n6:/tmp/pycore.39263/n6.conf# ip route add 192.168.31.128/27 via 10.31.0.1
root@n6:/tmp/pycore.39263/n6.conf# netstat -rn
Kernel IP routing table

```

Destination	Gateway	Genmask	Flags	MSS	Window	irtt	Iface
10.31.0.0	0.0.0.0	255.255.255.252	U	0	0	0	eth0
10.31.0.4	0.0.0.0	255.255.255.252	U	0	0	0	eth1
10.31.0.8	10.31.0.6	255.255.255.252	UG	0	0	0	eth1
10.31.0.12	10.31.0.6	255.255.255.252	UG	0	0	0	eth1
10.31.0.16	10.31.0.6	255.255.255.252	UG	0	0	0	eth1
10.31.0.20	10.31.0.6	255.255.255.252	UG	0	0	0	eth1
10.31.0.24	10.31.0.6	255.255.255.252	UG	0	0	0	eth1
10.31.0.28	10.31.0.6	255.255.255.252	UG	0	0	0	eth1
172.0.0.0	10.31.0.6	255.0.0.0	UG	0	0	0	eth1
172.16.142.0	10.31.0.1	255.255.255.252	UG	0	0	0	eth0
172.16.142.4	10.31.0.1	255.255.255.252	UG	0	0	0	eth0
172.16.143.0	10.31.0.6	255.255.255.252	UG	0	0	0	eth1
172.16.143.4	10.31.0.6	255.255.255.252	UG	0	0	0	eth1
192.168.31.128	10.31.0.1	255.255.255.224	UG	0	0	0	eth0
192.168.31.224	10.31.0.6	255.255.255.248	UG	0	0	0	eth1
192.168.31.232	10.31.0.6	255.255.255.248	UG	0	0	0	eth1
192.168.31.240	10.31.0.6	255.255.255.248	UG	0	0	0	eth1
192.168.31.248	10.31.0.6	255.255.255.248	UG	0	0	0	eth1

```

root@n6:/tmp/pycore.39263/n6.conf#

```

Figura 38: Tabela de encaminhamento do n6 otimizada com a rota sumarizada (/27).

Como evidenciado na Figura 38, a tabela do n6 ficou consideravelmente mais reduzida e limpa, substituindo quatro entradas por apenas uma.

Verificação de Conectividade: Para provar que a agregação de rotas não quebrou o roteamento, realizaram-se testes exaustivos de ping entre os vários polos da topologia.

```

vcmd
root@AfonsoHenriques:/tmp/pycore.39263/AfonsoHenriques.conf# traceroute -I 192.168.31.130
traceroute to 192.168.31.130 (192.168.31.130), 30 hops max, 60 byte packets
 1 192.168.31.225 (192.168.31.225) 0.033 ms 0.005 ms 0.003 ms
 2 172.16.143.1 (172.16.143.1) 0.013 ms 0.005 ms 0.006 ms
 3 10.31.0.29 (10.31.0.29) 0.017 ms 0.007 ms 0.007 ms
 4 10.31.0.25 (10.31.0.25) 0.017 ms 0.009 ms 0.010 ms
 5 10.31.0.13 (10.31.0.13) 0.020 ms 0.011 ms 0.011 ms
 6 10.31.0.17 (10.31.0.17) 0.024 ms 0.035 ms 0.014 ms
 7 10.31.0.5 (10.31.0.5) 0.024 ms 0.015 ms 0.015 ms
 8 10.31.0.1 (10.31.0.1) 0.023 ms 0.017 ms 0.017 ms
 9 172.16.142.2 (172.16.142.2) 0.028 ms 0.033 ms 0.019 ms
10 192.168.31.130 (192.168.31.130) 0.029 ms 0.020 ms 0.020 ms
root@AfonsoHenriques:/tmp/pycore.39263/AfonsoHenriques.conf# traceroute -I 192.168.31.146
traceroute to 192.168.31.146 (192.168.31.146), 30 hops max, 60 byte packets
 1 192.168.31.225 (192.168.31.225) 0.031 ms 0.004 ms 0.003 ms
 2 172.16.143.1 (172.16.143.1) 0.014 ms 0.006 ms 0.005 ms
 3 10.31.0.29 (10.31.0.29) 0.017 ms 0.007 ms 0.008 ms
 4 10.31.0.25 (10.31.0.25) 0.017 ms 0.009 ms 0.008 ms
 5 10.31.0.21 (10.31.0.21) 0.018 ms 0.011 ms 0.011 ms
 6 10.31.0.17 (10.31.0.17) 0.019 ms 0.026 ms 0.013 ms
 7 10.31.0.5 (10.31.0.5) 0.020 ms 0.014 ms 0.014 ms
 8 10.31.0.1 (10.31.0.1) 0.041 ms 0.023 ms 0.016 ms
 9 172.16.142.6 (172.16.142.6) 0.026 ms 0.018 ms 0.018 ms
10 192.168.31.146 (192.168.31.146) 0.028 ms 0.020 ms 0.019 ms
root@AfonsoHenriques:/tmp/pycore.39263/AfonsoHenriques.conf#

```

Figura 39: Testes de conectividade do Condado (AfonsoHenriques) para Galiza e CDN.

```

vcmd
root@Teresa:/tmp/pycore.39263/Teresa.conf# traceroute -I 192.168.31.226
traceroute to 192.168.31.226 (192.168.31.226), 30 hops max, 60 byte packets
 1 192.168.31.129 (192.168.31.129) 0.058 ms 0.007 ms 0.003 ms
 2 172.16.142.1 (172.16.142.1) 0.014 ms 0.005 ms 0.006 ms
 3 10.31.0.2 (10.31.0.2) 0.017 ms 0.005 ms 0.005 ms
 4 10.31.0.6 (10.31.0.6) 0.042 ms 0.010 ms 0.007 ms
 5 10.31.0.18 (10.31.0.18) 0.019 ms 0.025 ms 0.014 ms
 6 10.31.0.14 (10.31.0.14) 0.027 ms 0.023 ms 0.011 ms
 7 10.31.0.26 (10.31.0.26) 0.046 ms 0.015 ms 0.013 ms
 8 10.31.0.30 (10.31.0.30) 0.021 ms 0.014 ms 0.014 ms
 9 172.16.143.2 (172.16.143.2) 0.023 ms 0.015 ms 0.015 ms
10 192.168.31.226 (192.168.31.226) 0.025 ms 0.019 ms 0.016 ms
root@Teresa:/tmp/pycore.39263/Teresa.conf# traceroute -I 192.168.31.234
traceroute to 192.168.31.234 (192.168.31.234), 30 hops max, 60 byte packets
 1 192.168.31.129 (192.168.31.129) 0.026 ms 0.004 ms 0.003 ms
 2 172.16.142.1 (172.16.142.1) 0.012 ms 0.004 ms 0.004 ms
 3 10.31.0.2 (10.31.0.2) 0.012 ms 0.005 ms 0.006 ms
 4 10.31.0.6 (10.31.0.6) 0.014 ms 0.007 ms 0.007 ms
 5 10.31.0.10 (10.31.0.10) 0.018 ms 0.009 ms 0.008 ms
 6 10.31.0.14 (10.31.0.14) 0.016 ms 0.021 ms 0.010 ms
 7 10.31.0.26 (10.31.0.26) 0.017 ms 0.011 ms 0.012 ms
 8 10.31.0.30 (10.31.0.30) 0.018 ms 0.012 ms 0.012 ms
 9 172.16.143.6 (172.16.143.6) 0.755 ms 0.080 ms 0.034 ms
10 192.168.31.234 (192.168.31.234) 0.135 ms 0.034 ms 0.020 ms
root@Teresa:/tmp/pycore.39263/Teresa.conf# traceroute -I 192.168.31.146
traceroute to 192.168.31.146 (192.168.31.146), 30 hops max, 60 byte packets
 1 192.168.31.129 (192.168.31.129) 0.029 ms 0.004 ms 0.003 ms
 2 172.16.142.1 (172.16.142.1) 0.013 ms 0.005 ms 0.005 ms
 3 172.16.142.6 (172.16.142.6) 0.016 ms 0.006 ms 0.006 ms
 4 192.168.31.146 (192.168.31.146) 0.047 ms 0.012 ms 0.008 ms
root@Teresa:/tmp/pycore.39263/Teresa.conf#

```

Figura 40: Testes de conectividade da Galiza (Teresa) para o Condado, Institucional e CDN.

```

vcmd
root@Twitch:/tmp/pycore.39263/Twitch.conf# traceroute -I 192.168.31.226
traceroute to 192.168.31.226 (192.168.31.226), 30 hops max, 60 byte packets
 1 192.168.31.145 (192.168.31.145) 2.742 ms 0.018 ms 0.003 ms
 2 172.16.142.5 (172.16.142.5) 0.014 ms 0.005 ms 0.005 ms
 3 10.31.0.2 (10.31.0.2) 0.063 ms 0.008 ms 0.006 ms
 4 10.31.0.6 (10.31.0.6) 0.017 ms 0.007 ms 0.007 ms
 5 10.31.0.18 (10.31.0.18) 0.017 ms 0.009 ms 0.007 ms
 6 10.31.0.22 (10.31.0.22) 0.016 ms 0.051 ms 0.010 ms
 7 10.31.0.26 (10.31.0.26) 0.021 ms 0.012 ms 0.013 ms
 8 10.31.0.30 (10.31.0.30) 0.019 ms 0.013 ms 0.012 ms
 9 172.16.143.2 (172.16.143.2) 0.065 ms 0.018 ms 0.035 ms
10 192.168.31.226 (192.168.31.226) 0.045 ms 0.019 ms 0.018 ms
root@Twitch:/tmp/pycore.39263/Twitch.conf# traceroute -I 192.168.31.130
traceroute to 192.168.31.130 (192.168.31.130), 30 hops max, 60 byte packets
 1 192.168.31.145 (192.168.31.145) 0.027 ms 0.004 ms 0.003 ms
 2 172.16.142.5 (172.16.142.5) 0.013 ms 0.005 ms 0.004 ms
 3 172.16.142.2 (172.16.142.2) 0.015 ms 0.006 ms 0.006 ms
 4 192.168.31.130 (192.168.31.130) 0.015 ms 0.007 ms 0.007 ms
root@Twitch:/tmp/pycore.39263/Twitch.conf# traceroute -I 192.168.31.252
traceroute to 192.168.31.252 (192.168.31.252), 30 hops max, 60 byte packets
 1 192.168.31.145 (192.168.31.145) 0.037 ms 0.004 ms 0.003 ms
 2 172.16.142.5 (172.16.142.5) 0.017 ms 0.005 ms 0.005 ms
 3 10.31.0.2 (10.31.0.2) 0.016 ms 0.006 ms 0.007 ms
 4 10.31.0.6 (10.31.0.6) 0.016 ms 0.008 ms 0.008 ms
 5 10.31.0.10 (10.31.0.10) 0.019 ms 0.010 ms 0.010 ms
 6 10.31.0.22 (10.31.0.22) 0.052 ms 0.036 ms 0.012 ms
 7 10.31.0.26 (10.31.0.26) 0.021 ms 0.014 ms 0.014 ms
 8 10.31.0.30 (10.31.0.30) 0.022 ms 0.016 ms 0.015 ms
 9 172.16.143.6 (172.16.143.6) 0.093 ms 0.025 ms 0.019 ms
10 192.168.31.252 (192.168.31.252) 0.091 ms 0.023 ms 0.021 ms
root@Twitch:/tmp/pycore.39263/Twitch.conf#

```

Figura 41: Testes de conectividade da CDN (Twitch) para o Condado e Galiza.

A análise das capturas (Figura 39, Figura 40 e Figura 41) comprova que não houve qualquer perda de pacotes nas comunicações. A conectividade total foi mantida com sucesso, concluindo-se que a decisão de encaminhamento no nó se tornou mais célere e eficiente, libertando memória na tabela de roteamento do dispositivo.

4. Conclusão

O presente trabalho prático cumpriu integralmente os objetivos propostos, consolidando o estudo detalhado do protocolo IPv4.

Através da análise de tráfego, do planeamento de sub-redes (Subnetting) e da otimização de caminhos (Supernetting), o grupo foi capaz de diagnosticar, justificar e corrigir uma série de anomalias de encaminhamento propositadamente introduzidas numa infraestrutura simulada. As restrições do endereçamento IPv4 e a necessidade da sua gestão criteriosa ficaram evidentes, preparando o terreno para a futura compreensão de protocolos de nova geração, como o IPv6.

Conclui-se, assim, que a robustez de uma rede não provém apenas da interligação física dos seus equipamentos, mas sim da coerência, simetria e otimização das suas tabelas de encaminhamento lógico.